BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN TRÍ TUỆ NHÂN TẠO

Học phần AIP202 · 2 tín chỉ

AI ĐIỀU PHỐI VÀ GHÉP CHUYẾN XE CÁ NHÂN

Sinh viên thực hiện : **Trương Thị Ngọc Hậu - Cao Tấn Đạt (2351220190 - 2351220191)**

Lớp : **23CT4**

Ngành : **Công nghệ thông tin**

Giảng viên hướng dẫn : **TS. Đỗ Phúc Hảo**

Năm học : **2025–2026**

Đà Nẵng · 2025–2026

Lời cam đoan

Tôi xin cam đoan đồ án này là công trình do chính tôi thực hiện dưới sự hướng dẫn của TS. Đỗ Phúc Hảo. Các nội dung nghiên cứu, số liệu và kết quả trình bày trong đồ án là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác.

Mọi số liệu, hình ảnh, mã nguồn và kết quả của tác giả khác được sử dụng trong đồ án đều được trích dẫn nguồn gốc rõ ràng trong phần tài liệu tham khảo. Toàn bộ phần mềm và thư viện được dùng đều có giấy phép cho phép sử dụng. Những phần có sử dụng công cụ trí tuệ nhân tạo hỗ trợ đã được nêu rõ trong báo cáo.

Nếu phát hiện có bất kỳ sự gian lận nào, tôi xin chịu hoàn toàn trách nhiệm trước Khoa và Nhà trường.

Đà Nẵng, ngày ... tháng ... năm ...
Sinh viên thực hiện

Trương Thị Ngọc Hậu - Cao Tấn Đạt

Lời cảm ơn

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Đỗ Phúc Hảo– người đã trực tiếp định hướng, tận tình chỉ bảo và truyền đạt những kiến thức chuyên môn quý báu cho chúng em trong suốt quá trình thực hiện đề tài. Sự đồng hành, những lời góp ý sâu sắc và sự động viên của Thầy chính là động lực lớn giúp chúng em từng bước hoàn thiện nghiên cứu này. Dù đã cố gắng nỗ lực hết sức, song do kiến thức và kinh nghiệm thực tế của chúng em còn hạn chế nên đồ án khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được những ý kiến đóng góp, nhận xét từ Thầy đánh giá để đề tài được hoàn thiện hơn. Chúng em xin kính chúc TS. Đỗ Phúc Hảoluôn dồi dào sức khỏe, may mắn và thành công trong sự nghiệp trồng người. Chúng em xin chân thành cảm ơn!

Đà Nẵng, ngày ... tháng ... năm ...
Sinh viên thực hiện

Trương Thị Ngọc Hậu - Cao Tấn Đạt

Tóm tắt

Trong bối cảnh đô thị hóa nhanh chóng, nhu cầu di chuyển cá nhân tăng cao dẫn đến tình trạng ùn tắc giao thông và ô nhiễm môi trường nghiêm trọng. Vấn đề cốt lõi cần giải quyết là làm sao để tối ưu hóa việc ghép các chuyến xe cá nhân đi chung tuyến đường nhằm giảm lượng phương tiện lưu thông và chi phí di chuyển. Để giải quyết bài toán này, đề tài ứng dụng thuật toán Dijkstra kết hợp Học máy / thuật toán di truyền Genetic Algorithm nhằm tìm kiếm và ghép nối các hành trình tương đồng theo thời gian thực. Kết quả thực nghiệm cho thấy hệ thống đã tối ưu hóa tỷ lệ ghép chuyến thành công đạt , giảm trung bình quãng đường di chuyển thừa và tiết kiệm thời gian chờ đợi của hành khách. Nghiên cứu này mang ý nghĩa thiết thực trong việc nâng cao hiệu quả giao thông đô thị, hỗ trợ phát triển các mô hình di chuyển thông minh và giảm thiểu xả thải khí carbon ra môi trường

Mục lục

Lời cam đoan i

Lời cảm ơn ii

Tóm tắt iii

Danh mục giải thuật ix

1 Mở đầu 1
- 1.1 Lý do chọn đề tài . . . 1
- 1.2 Mục tiêu của đồ án . . . 1
- 1.3 Phạm vi và đối tượng . . . 2
  - 1.3.1 Phạm vi nghiên cứu . . . 2
  - 1.3.2 Đối tượng sử dụng . . . 3
- 1.4 Bố cục báo cáo . . . 3

2 Cơ sở lý thuyết và công nghệ 4
- 2.1 Kiến thức nền tảng . . . 4
  - 2.1.1 Xử lý ngôn ngữ tự nhiên (NLP) và Mô hình ngôn ngữ lớn (LLM) . . . 4
  - 2.1.2 Kỹ thuật thu thập dữ liệu web (Web Crawling) . . . 7
  - 2.1.3 Hệ thống Thông tin Địa lý (Geocoding & Haversine) . . . 8
- 2.2 Công nghệ và thư viện sử dụng . . . 11
  - 2.2.1 Backend (Xử lý dữ liệu và Động cơ AI) . . . 11
  - 2.2.2 Frontend (Giao diện Dashboard Quản trị và Cổng người dùng) . . . 15
- 2.3 Các công trình liên quan và Tính sáng tạo của đề tài . . . 15

3 Phân tích và thiết kế hệ thống 18
- 3.1 Phân tích bài toán và yêu cầu hệ thống . . . 18
  - 3.1.1 Phân tích bài toán thực tế . . . 18
  - 3.1.2 Phân tích yêu cầu chức năng cụ thể . . . 19
  - 3.1.3 Phân tích yêu cầu phi chức năng . . . 19
- 3.2 Thiết kế kiến trúc và luồng xử lý hệ thống . . . 20

3.2.1 Sơ đồ phân rã chức năng (Functional Decomposition Diagram) . . . 20
3.2.2 Sơ đồ luồng xử lý tổng thể hệ thống (End-to-End System Flow) . . . 21
3.2.3 Sơ đồ ca sử dụng (Use Case Diagram) . . . 24
3.2.4 Sơ đồ trình tự (Sequence Diagram) . . . 25
3.3 Thiết kế cơ sở dữ liệu . . . 26
3.3.1 Sơ đồ thực thể – mối quan hệ (ERD) . . . 26
3.3.2 Thiết kế chi tiết các bảng dữ liệu . . . 28
3.4 Thiết kế mô hình AI và giải thuật điều phối . . . 28
3.4.1 Thiết kế mô hình NLP/LLM trích xuất thông tin hành trình 28
3.4.2 Thiết kế giải thuật điều phối ghép chuyến tối ưu . . . 28

4 Cài đặt và thử nghiệm hệ thống 31
4.1 Môi trường phát triển và Triển khai hạ tầng . . . 31
4.1.1 Môi trường phần cứng và hạ tầng đám mây . . . 31
4.1.2 Ngăn xếp công nghệ và cấu trúc mã nguồn . . . 32
4.2 Cài đặt các mô-đun chức năng cốt lõi . . . 32
4.2.1 Mô-đun thu thập dữ liệu tự động (Multi-Platform Crawler) 32
4.2.2 Mô-đun tiền xử lý và Bóc tách thực thể AI (NLP / LLM Engine) . . . 32
4.2.3 Mô-đun định vị tọa độ và Điều phối ghép chuyến . . . 34
4.3 Kết quả triển khai giao diện các phân hệ người dùng . . . 34
4.3.1 Giao diện Trang chủ công cộng và Đăng ký phân quyền . 35
4.3.2 Giao diện phân hệ Tài xế (Driver Dashboard) . . . 36
4.3.3 Giao diện phân hệ Hành khách (Customer Dashboard) . . 38

5 Kết quả và đánh giá 41
5.1 Kết quả vận hành và Giám sát dữ liệu trên hệ thống quản trị . . 41
5.1.1 Kết quả thu thập và quản lý kho bài viết thô từ mạng xã hội 41
5.1.2 Thống kê tổng quan hiệu suất AI và phân bổ nguồn dữ liệu 42
5.1.3 Phân tích mật độ địa điểm và tuyến đường phổ biến . . . 43
5.2 Độ đo đánh giá và Thống kê thực nghiệm . . . 44
5.2.1 Kịch bản thử nghiệm và công thức độ đo . . . 44
5.2.2 Đánh giá chất lượng bóc tách thông tin NLP . . . 45
5.2.3 Đánh giá hiệu năng thuật toán ghép chuyến và Thời gian phản hồi . . . 46
5.2.4 So sánh hiệu quả giữa LLM Engine và phương pháp truyền thống . . . 46

5.3 Bàn luận, Ưu điểm và Hạn chế 46
5.3.1 Bàn luận kết quả thực nghiệm 46
5.3.2 Đánh giá ưu điểm và hạn chế 49

6 Kết luận và hướng phát triển 50
6.1 Kết luận 50
6.2 Bài học rút ra 51
6.3 Hướng phát triển trong tương lai 52
6.3.1 Hoàn thiện hạ tầng Kỹ thuật và AI 52
6.3.2 Mở rộng Tính năng và Trải nghiệm Người dùng 52

Tài liệu tham khảo 53

Danh sách hình vẽ

3.1 Sơ đồ phân rã chức năng của hệ thống AI điều phối và ghép chuyến xe cá nhân. 21
3.2 Sơ đồ luồng xử lý dữ liệu tổng thể từ thu thập đa kênh đến điều phối ghép chuyến. 23
3.3 Sơ đồ Use Case tổng quan thể hiện tương tác của 4 tác nhân chính với hệ thống. 24
3.4 Sơ đồ trình tự biểu diễn luồng xử lý dữ liệu từ lúc thu thập đến khi hiển thị kết quả. 25
3.5 Sơ đồ thực thể – mối quan hệ (ERD) của cơ sở dữ liệu `xeghep_db`. 27

4.1 Giao diện Trang chủ công cộng của nền tảng Xe Tiện Chuyến. . . 35
4.2 Giao diện Đăng ký tài khoản mới và phân quyền vai trò người dùng. 36
4.3 Giao diện Không gian làm việc Tổng quan dành cho Tài xế. . . . 37
4.4 Giao diện Quản lý danh sách Chuyến xe của Tài xế. 38
4.5 Giao diện Không gian làm việc Tổng quan dành cho Hành khách. 38
4.6 Giao diện Tìm kiếm chuyến xe tiện chuyến dành cho Hành khách. 39
4.7 Giao diện Theo dõi trạng thái chuyến đi và Đánh giá chất lượng tài xế. 40

5.1 Giao diện Quản lý Bài viết thô thu thập tự động từ mạng xã hội. 42
5.2 Giao diện Thống kê tổng quan hiệu suất AI và phân bổ nguồn dữ liệu. 43
5.3 Giao diện Thống kê mật độ Địa điểm xuất phát, Điểm đến và Tuyến đường phổ biến. 44

Danh sách bảng

2.1 Đặc tả chi tiết các trường thực thể NER và quy tắc chuẩn hóa dữ liệu từ văn bản tiếng Việt. . . . 6
2.2 Phân tích đặc điểm kỹ thuật và cơ chế thu thập dữ liệu trên từng nền tảng mạng xã hội. . . . 9
2.3 Giải thích các đại lượng toán học và ngưỡng ràng buộc trong bài toán định vị – ghép chuyến. . . . 12
2.4 Tổng hợp ngăn xếp công nghệ, thư viện và vai trò trong phân hệ Backend. . . . 14
2.5 Đặc tả các thành phần giao diện và thư viện sử dụng trong phân hệ Frontend. . . . 16
2.6 Bảng so sánh đối sánh giữa các nền tảng gọi xe truyền thống và Đề tài. . . . 17

3.1 Thống kê tiêu chí phân tích yêu cầu hệ thống điều phối xe ghép. . 20
3.2 Bảng đặc tả chi tiết các chức năng cụ thể của hệ thống theo từng phân hệ. . . . 22
3.3 Mô tả thuộc tính chi tiết các bảng trong cơ sở dữ liệu `xeghep_db`. 29
3.4 Quy định trọng số và giá trị ngưỡng trong giải thuật ghép chuyến AI. . . . 30

4.1 Tổng hợp thông số môi trường cài đặt và ngăn xếp công nghệ triển khai hệ thống. . . . 33

5.1 Thống kê độ chính xác bóc tách các thực thể thông tin hành trình của mô-đun AI. . . . 45
5.2 Hiệu năng thuật toán điều phối ghép chuyến theo quy mô bản ghi cơ sở dữ liệu. . . . 46
5.3 Đánh giá thời gian phản hồi (*Latency*) và tỷ lệ lỗi của từng mô-đun hệ thống. . . . 47
5.4 So sánh hiệu quả trích xuất thông tin giữa LLM Engine và Biểu thức chính quy (Regex). . . . 48

DANH MỤC GIẢI THUẬT

1 Trích xuất thực thể hành trình tiếng Việt bằng LLM và Regex dự phòng 7
2 Thu thập dữ liệu web đa nền tảng qua HTTP Session và khử trùng lặp 10
3 Tính khoảng cách cầu Haversine và chấm điểm ghép chuyến không gian – thời gian 13

1

Mở đầu

1.1 Lý do chọn đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của các nền tảng mạng xã hội như Facebook, Zalo và Threads đã tạo ra một lượng lớn dữ liệu phản ánh trực tiếp nhu cầu di chuyển của cộng đồng, đặc biệt là nhu cầu tìm xe tiện chuyến, xe ghép (*carpooling*) trên các tuyến đường liên tỉnh nhằm tiết kiệm chi phí và giảm thiểu mật độ phương tiện, ùn tắc giao thông. Tuy nhiên, phần lớn quá trình tìm kiếm và ghép nối giữa hành khách với tài xế hiện nay vẫn diễn ra hoàn toàn thủ công. Người dùng phải tự đăng bài và đọc lướt từng bài viết, bình luận trong hàng loạt hội nhóm khác nhau, dẫn đến tốn nhiều thời gian, dễ bỏ lỡ chuyến đi phù hợp do bài đăng trôi nhanh, đồng thời gặp khó khăn trong việc đối chiếu lộ trình và xác định độ tin cậy của thông tin.

Bên cạnh đó, các bài đăng trên mạng xã hội có tính phi cấu trúc cao, thường xuyên sử dụng từ viết tắt địa phương, tiếng lóng giao thông và cách diễn đạt thời gian không theo quy chuẩn, khiến các công cụ tìm kiếm từ khóa thông thường kém hiệu quả. Xuất phát từ thực trạng đó, việc ứng dụng Trí tuệ nhân tạo (AI) để tự động hóa toàn bộ quy trình từ thu thập dữ liệu đa kênh, phân tích ngữ nghĩa tiếng Việt bằng Mô hình ngôn ngữ lớn đến tối ưu hóa điều phối lộ trình không gian – thời gian là vô cùng cần thiết. Do đó, nhóm chúng em quyết định lựa chọn đề tài **"AI điều phối và ghép chuyến xe cá nhân"** nhằm xây dựng một hệ thống thông minh giúp kết nối tài xế và hành khách một cách nhanh chóng, chính xác và tối ưu nhất.

1.2 Mục tiêu của đồ án

Đồ án hướng tới các mục tiêu cụ thể sau:

- Tự động hóa quá trình thu thập dữ liệu (*crawling*) đa kênh các bài đăng tìm

xe, ghép xe từ mạng xã hội (Facebook, Zalo, Threads) theo chu kỳ định kỳ kết hợp cơ chế khử trùng lặp nội dung.

- Ứng dụng công nghệ Xử lý ngôn ngữ tự nhiên (NLP) thông qua Mô hình ngôn ngữ lớn (LLM – Google Gemini, Groq LLaMA, OpenAI) kết hợp bộ lọc biểu thức chính quy (Regex) dự phòng để nhận diện vai trò người đăng và trích xuất chính xác các thực thể cốt lõi (điểm đi, điểm đến, thời gian khởi hành, số ghế trống/số hành khách, số điện thoại liên hệ) từ văn bản phi cấu trúc tiếng Việt sang định dạng JSON chuẩn.
- Xây dựng giải thuật quy đổi tọa độ địa lý (*Geocoding*), tính toán khoảng cách cầu Haversine và lọc ràng buộc không gian – thời gian ($\Delta D \leq 5{,}0$ km, $\Delta T \leq 60$ phút) cùng hàm chấm điểm đa tiêu chí để đề xuất các cặp ghép chuyến phù hợp nhất cho người dùng.
- Thiết kế và triển khai ứng dụng Web hoàn chỉnh theo kiến trúc Client – Server (FastAPI Backend và React 19 Frontend) tại địa chỉ `https://goghep.id.vn` với giao diện trực quan, phân quyền rõ ràng giúp người dùng và quản trị viên dễ dàng vận hành, quản lý chuyến đi.

1.3 Phạm vi và đối tượng

1.3.1 Phạm vi nghiên cứu

- **Dữ liệu đầu vào:** Các bài đăng công khai bằng tiếng Việt tự do trên các hội nhóm Facebook, Zalo và Threads liên quan đến nhu cầu đi chung xe, kết hợp dữ liệu người dùng nhập trực tiếp thông qua biểu mẫu trên cổng ứng dụng Web.
- **Phạm vi địa lý:** Tập trung thử nghiệm và đánh giá trên các tuyến đường liên tỉnh phổ biến tại Việt Nam (khu vực Miền Bắc như Hà Nội – Hải Phòng – Quảng Ninh – Nam Định – Thái Bình và khu vực Miền Trung như Đà Nẵng – Quảng Nam – Quảng Ngãi – Thừa Thiên Huế).
- **Phạm vi tuân thủ pháp lý:** Thiết kế quy trình ẩn danh hóa định danh mạng xã hội và xác thực tài khoản người dùng qua số điện thoại theo định hướng của Nghị định số 13/2023/NĐ-CP về Bảo vệ dữ liệu cá nhân [5] và Nghị định số 147/2024/NĐ-CP về Quản lý, cung cấp, sử dụng dịch vụ Internet và thông tin trên mạng [14].

1.3.2 Đối tượng sử dụng

- *Hành khách (`khach_hang`):* Những cá nhân có nhu cầu tìm xe ghép, xe tiện chuyến liên tỉnh hoặc đi sân bay với chi phí tiết kiệm, có thể đăng yêu cầu tìm xe hoặc tra cứu trực tiếp các chuyến xe phù hợp do hệ thống đề xuất.
- *Tài xế (`tai_xe`):* Các chủ xe cá nhân hoặc nhà xe tuyến cố định có ghế trống muốn tìm thêm hành khách thuận đường để tối ưu hóa doanh thu và chi phí vận hành chuyến đi.
- *Quản trị viên hệ thống (`admin`):* Người vận hành nền tảng, chịu trách nhiệm điều khiển các tiến trình thu thập dữ liệu, giám sát hiệu năng trích xuất AI, kiểm duyệt dữ liệu và theo dõi thống kê tuyến đường.

1.4 Bố cục báo cáo

Báo cáo đồ án được tổ chức thành 5 chương chính:

- **Chương 1 – Mở đầu:** Trình bày lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi, đối tượng sử dụng và bố cục tổng thể của báo cáo.
- **Chương 2 – Cơ sở lý thuyết và công nghệ nền tảng:** Trình bày các kiến thức nền tảng về NLP/NER, Mô hình ngôn ngữ lớn (LLM), kỹ thuật thu thập dữ liệu web, định vị địa lý Geocoding, công thức Haversine và ngăn xếp công nghệ sử dụng trong đồ án.
- **Chương 3 – Phân tích và thiết kế hệ thống:** Phân tích yêu cầu, trình bày kiến trúc tổng thể, hệ thống sơ đồ chức năng, hành vi, thiết kế cơ sở dữ liệu quan hệ và các giải thuật điều phối cốt lõi.
- **Chương 4 – Cài đặt và triển khai hệ thống:** Mô tả cấu trúc mã nguồn, chi tiết cài đặt các mô-đun xử lý AI, lập lịch tự động và triển khai hệ thống thực tế trên hạ tầng đám mây.
- **Chương 5 – Thử nghiệm, kết quả và đánh giá:** Trình bày các giao diện vận hành thực tế, phân tích định lượng độ chính xác mô-đun AI, hiệu năng giải thuật ghép chuyến và đánh giá ưu, nhược điểm.
- **Kết luận và hướng phát triển:** Tổng kết những kết quả đạt được, bài học kinh nghiệm và đề xuất hướng mở rộng hệ thống trong tương lai.

2

Cơ sở lý thuyết và công nghệ

Mục tiêu của chương: *Cung cấp hệ thống kiến thức nền tảng về Trí tuệ nhân tạo, Xử lý ngôn ngữ tự nhiên (NLP), Mô hình ngôn ngữ lớn (LLM), kỹ thuật thu thập dữ liệu web tự động và bài toán tối ưu không gian – thời gian trong điều phối vận tải. Đồng thời, chương này trình bày chi tiết các giải thuật cốt lõi, tổng hợp ngăn xếp công nghệ, thư viện sử dụng và đối sánh với các giải pháp hiện có trên thị trường nhằm làm nổi bật tính cấp thiết và điểm sáng tạo của đề tài.*

2.1 Kiến thức nền tảng

2.1.1 Xử lý ngôn ngữ tự nhiên (NLP) và Mô hình ngôn ngữ lớn (LLM)

Xử lý ngôn ngữ tự nhiên (*Natural Language Processing – NLP*) là một nhánh trọng yếu của Trí tuệ nhân tạo (AI) tập trung nghiên cứu và phát triển các mô hình tính toán giúp máy tính có khả năng đọc hiểu, phân tích ngữ nghĩa, diễn dịch và sinh ra ngôn ngữ tự nhiên của con người [6]. Trong phạm vi đề tài điều phối xe tiện chuyến, bài toán cốt lõi cần giải quyết thuộc nhóm **Trích xuất thông tin (*Information Extraction – IE*)** và **Nhận dạng thực thể có tên (*Named Entity Recognition – NER*)** [8]. Mục tiêu kỹ thuật là chuyển đổi các chuỗi văn bản thô phi cấu trúc thu thập từ mạng xã hội thành đối tượng dữ liệu JSON có lược đồ cố định để phục vụ lưu trữ quan hệ và tính toán tối ưu.

Thực tế khảo sát cho thấy các bài đăng trên hội nhóm xe ghép bằng tiếng Việt tự do mang nhiều đặc thù ngôn ngữ phức tạp, gây trở ngại lớn cho các hệ thống phân tích văn bản truyền thống:

- **Từ viết tắt và ký hiệu địa phương dày đặc:** Người dùng thường xuyên rút gọn tên tỉnh thành, quận huyện hoặc điểm mốc giao thông (ví dụ: *"hn"* – Hà Nội, *"sg"* – TP. Hồ Chí Minh, *"đn"* – Đà Nẵng, *"nhs"* – Ngũ Hành Sơn,

"sb nb" – Sân bay Nội Bài).

- **Tiếng lóng và thuật ngữ đặc thù ngành vận tải ghép:** Các cụm từ như *"mống"* (chỉ số lượng hành khách), *"ghế trống"* (số chỗ còn nhận thêm), *"xe cọc"* hoặc *"xe quay đầu"* (xe chạy chiều về cần tìm khách tiện chuyến), *"bao xe"* (thuê trọn gói cả chuyến) xuất hiện với tần suất cao nhưng không nằm trong từ điển tiếng Việt chuẩn.

- **Biểu diễn thời gian tương đối và cấu trúc câu tự do:** Mốc giờ khởi hành thường viết theo ngôn ngữ nói (như *"sáng mai tầm 7h30"*, *"chiều nay quay đầu"*), câu thiếu chủ ngữ – vị ngữ, mắc lỗi chính tả, thiếu dấu thanh hoặc chèn xen kẽ nhiều biểu tượng cảm xúc (*emoji*) và nội dung quảng cáo.

Các phương pháp trích xuất thế hệ trước dựa trên luật từ điển (*Rule-based*), biểu thức chính quy (*Regex*) hay các mô hình học sâu tuần tự như BiLSTM-CRF [8] đòi hỏi phải xây dựng tập luật thủ công đồ sộ, tốn nhiều công sức gán nhãn dữ liệu huấn luyện và dễ suy giảm độ chính xác khi gặp biến thể viết tắt mới.

Do đó, đồ án ứng dụng sức mạnh của các **Mô hình ngôn ngữ lớn (*Large Language Models – LLM*)** như Google Gemini [11] và Groq LLaMA [9] thông qua kỹ thuật **Prompt Engineering**. Nhờ kiến trúc Transformer với cơ chế tự chú ý đa đầu (*Multi-Head Self-Attention*) được tiền huấn luyện trên kho ngữ liệu đa ngôn ngữ quy mô lớn [6, 7], LLM có khả năng suy luận ngữ cảnh sâu sắc và học từ ít ví dụ (*Few-shot Prompting*) mà không cần huấn luyện lại trọng số. Bằng cách thiết kế câu lệnh định hướng (*System & User Prompts*) kèm ràng buộc định dạng đầu ra JSON, LLM thực hiện đồng thời bốn nhiệm vụ:

- **Lọc nhiễu và phân loại ý định:** Loại bỏ các bài đăng quảng cáo rác, tìm việc làm và xác định chính xác vai trò người đăng là *Tài xế tìm khách* (`tai_xe`) hay *Hành khách tìm xe* (`khach_hang`).

- **Bóc tách thực thể cốt lõi:** Nhận diện đầy đủ Điểm đón (*Origin*), Điểm đến (*Destination*), Mốc thời gian xuất phát (*Departure Time*), Số lượng ghế trống hoặc số hành khách và Số điện thoại liên hệ.

- **Chuẩn hóa dữ liệu hành chính:** Tự động giải nghĩa các từ viết tắt địa danh về tên đơn vị hành chính chuẩn (quận/huyện, tỉnh/thành phố) và chuẩn hóa số điện thoại về chuỗi 10 chữ số liên tục.

- **Tách đa hành trình trong một bài đăng:** Khi một tài xế đăng đồng thời cả lịch chiều đi và chiều về trong cùng một bài viết, mô hình tự động tách thành hai bản ghi chuyến xe độc lập.

Bảng 2.1. Đặc tả chi tiết các trường thực thể NER và quy tắc chuẩn hóa dữ liệu từ văn bản tiếng Việt.

Tên thực thể	**Khóa JSON**	**Biểu hiện thô trên MXH**	**Quy tắc chuẩn hóa đầu ra**
Vai trò người đăng	`vai_tro`	*"xe quay đầu"*, *"còn 2 ghế"*, *"tìm 1 mống"*, *"cần tìm xe ghép"*	Phân loại nhị phân về hai nhãn chuẩn: `tai_xe` hoặc `khach_hang`.
Điểm xuất phát	`diem_di`	*"hn"*, *"nhs đn"*, *"bigc thăng long"*, *"sb nb"*	Giải nghĩa viết tắt và bổ sung cấp tỉnh/thành phố (VD: *"Ngũ Hành Sơn, Đà Nẵng"*).
Điểm kết thúc	`diem_den`	*"sg"*, *"qng"*, *"tp nam định"*, *"hội an"*	Chuẩn hóa địa danh đích theo định dạng hành chính Việt Nam để phục vụ Geocoding.
Thời gian khởi hành	`thoi_gian`	*"7h30 sáng mai"*, *"15g chiều nay"*, *"chạy liên tục"*	Quy đổi về định dạng giờ chuẩn `HH:MM DD/MM` hoặc nhãn `Linh hoạt`.
Số ghế / Số khách	`so_luong`	*"2 mống"*, *"3 ghế trống"*, *"bao xe 4c"*	Trích xuất giá trị số nguyên dương (`INT`), mặc định bằng 1 nếu không nêu rõ.
Số điện thoại	`sdt`	*"0987.654.321"*, *"0987 654 321"*, *"+84987654321"*	Loại bỏ dấu chấm, khoảng trắng, quy đổi đầu số +84 thành chuỗi 10 chữ số `0xxxxxxxxx`.

Quy trình tiền xử lý văn bản, gọi Mô hình ngôn ngữ lớn kết hợp xoay vòng khóa API và cơ chế biểu thức chính quy (Regex) dự phòng được mô tả chi tiết trong **Giải thuật 1**.

Giải thuật 1. Trích xuất thực thể hành trình tiếng Việt bằng LLM và Regex dự phòng

Require: Chuỗi văn bản bài viết thô T, tập khóa API xoay vòng $\mathcal{K}_{\text{LLM}}$
Ensure: Đối tượng JSON thực thể $\mathcal{J}$ hoặc trạng thái loại bỏ `INVALID`

$T_{\text{clean}} \leftarrow \text{RemoveNoiseAndEmoji}(T)$ ▷ Làm sạch ký tự rác và khoảng trắng thừa
$\text{phone}_{\text{backup}} \leftarrow \text{RegexExtractPhone}(T_{\text{clean}})$ ▷ Bóc tách SĐT 10 số theo chuẩn Việt Nam
$\mathcal{J} \leftarrow \text{NULL}$
if $\mathcal{K}_{\text{LLM}} \neq \emptyset$ **then**
    Xây dựng câu lệnh $\text{Prompt}(T_{\text{clean}})$ theo kỹ thuật *Few-shot JSON Schema*
    for mỗi khóa $k \in \mathcal{K}_{\text{LLM}}$ theo cơ chế vòng tròn (*Round-Robin*) **do**
        $\text{raw_resp} \leftarrow \text{InvokeLLMAPI}(\text{Prompt}(T_{\text{clean}}), k)$
        if raw_resp hợp lệ **và** ValidateJSONSchema(raw_resp) **then**
            $\mathcal{J} \leftarrow \text{DecodeJSON}(\text{raw_resp})$
            thoát vòng lặp
        end if
    end for
end if
if $\mathcal{J} = \text{NULL}$ **then** ▷ Kích hoạt bộ phân tích luật dự phòng khi API quá tải
    $\mathcal{J} \leftarrow \text{RuleBasedRegexFallback}(T_{\text{clean}})$
end if
if $\mathcal{J}.\text{sdt} = \emptyset$ **then**
    $\mathcal{J}.\text{sdt} \leftarrow \text{phone}_{\text{backup}}$
end if
if $\mathcal{J}.\text{diem_di} = \emptyset$ **hoặc** $\mathcal{J}.\text{diem_den} = \emptyset$ **hoặc** $\mathcal{J}.\text{sdt} = \emptyset$ **then**
    return `INVALID` ▷ Bài viết thiếu thông tin hành trình tối thiểu
end if
return $\text{NormalizeAdministrativeNames}(\mathcal{J})$

2.1.2 Kỹ thuật thu thập dữ liệu web (Web Crawling)

Thu thập dữ liệu web (*Web Crawling / Web Scraping*) là quá trình tự động hóa việc gửi yêu cầu truy cập tới máy chủ web, phân tích cấu trúc tài liệu HTML/JSON trả về và lưu trữ thông tin mục tiêu vào kho dữ liệu [16]. Đối với các hệ thống

thu thập dữ liệu thời gian thực từ các nền tảng mạng xã hội lớn như Facebook, Threads hay Zalo, thách thức kỹ thuật lớn nhất bao gồm:

- **Cơ chế xác thực phiên và kiểm soát truy cập:** Phần lớn nội dung hội nhóm chỉ hiển thị khi người dùng đã đăng nhập, đồng thời máy chủ áp dụng cơ chế giới hạn tần suất truy cập (*Rate Limiting*) và phát hiện hành vi tự động hóa bất thường.
- **Cấu trúc giao diện động thay đổi liên tục:** Mã nguồn HTML và các lớp CSS của mạng xã hội thường xuyên được làm rối (*obfuscation*) hoặc thay đổi theo từng bản cập nhật.
- **Tài nguyên vận hành trên máy chủ đám mây:** Việc chạy đồng thời nhiều trình duyệt điều khiển tự động đầy đủ (như Selenium hay Playwright) tiêu tốn lượng lớn bộ nhớ RAM và CPU.

Để khắc phục các thách thức này, hệ thống áp dụng phương pháp thu thập dữ liệu nhẹ thông qua giao thức **HTTP Session Crawling** kết hợp giả lập định danh trình duyệt (`User-Agent`, `Accept-Language`) và mã phiên đăng nhập (`Cookie Injection`). Cách tiếp cận này giúp giảm hơn 80% mức tiêu thụ bộ nhớ RAM so với trình duyệt ảo hóa, tăng tốc độ tải trang và đảm bảo tính ổn định cho các tiến trình chạy ngầm định kỳ. **Bảng 2.2** phân tích chi tiết đặc điểm kỹ thuật thu thập dữ liệu trên ba nền tảng mạng xã hội được tích hợp trong hệ thống.

Quy trình thu thập dữ liệu đa kênh, kiểm soát tần suất và khử trùng lặp trước khi lưu vào bảng `bai_viet_tho` được trình bày tại **Giải thuật 2**.

2.1.3 Hệ thống Thông tin Địa lý (Geocoding & Haversine)

Geocoding là quá trình chuyển đổi địa chỉ dạng văn bản (*textual address*) thành cặp tọa độ địa lý thực tế trên hệ quy chiếu toàn cầu WGS84 bao gồm Vĩ độ (*Latitude*, φ) và Kinh độ (*Longitude*, λ) tính theo đơn vị độ, sau đó quy đổi sang đơn vị radian theo công thức $\text{rad} = \text{deg} \times \frac{\pi}{180}$. Việc ứng dụng API bản đồ (OpenStreetMap / Geopy) cho phép hệ thống số hóa không gian hành trình, từ đó tính toán chính xác độ lệch khoảng cách giữa điểm đón/trả của hành khách với lộ trình của tài xế.

Do bề mặt Trái Đất có dạng hình cầu, khoảng cách đường chim bay ngắn nhất giữa hai cặp tọa độ $P_1(\varphi_1, \lambda_1)$ và $P_2(\varphi_2, \lambda_2)$ được tính bằng **công thức Haversine** [12]:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1) \cdot \cos(\varphi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{2.1}$$

Bảng 2.2. Phân tích đặc điểm kỹ thuật và cơ chế thu thập dữ liệu trên từng nền tảng mạng xã hội.

Nền tảng	Mô-đun thực thi	Cơ chế xác thực & truy vấn	Chiến lược chống chặn & lọc nhiễu
Facebook	`crawl_facebook_http.py`	HTTP Session đính kèm `FB_COOKIE` và `User-Agent` di động/web; truy vấn theo danh sách ID hội nhóm xe ghép.	Giãn cách ngẫu nhiên 2–5 giây giữa các trang, loại bỏ bài ghim quảng cáo và kiểm tra trùng lặp nội dung.
Threads	`crawl_threads_http.py`	Gửi HTTP GET kèm `THREADS_COOKIE` tới các thẻ chủ đề (*tags*) và luồng tìm kiếm từ khóa xe tiện chuyến.	Phân tích cây DOM và khối dữ liệu JSON nhúng, lọc bỏ bài viết dưới 15 ký tự hoặc không chứa số điện thoại.
Zalo Web	`crawl_zalo_http.py`	Bắt buộc sử dụng phiên đăng nhập `ZALO_COOKIES` hợp lệ để truy xuất tin đăng từ các nhóm cộng đồng.	Tự động kiểm tra tính hợp lệ của phiên trước khi quét; ghi cảnh báo vào nhật ký xoay vòng nếu hết hạn Cookie.

Giải thuật 2. Thu thập dữ liệu web đa nền tảng qua HTTP Session và khử trùng lặp

Require: Tập nền tảng kích hoạt $\mathcal{S} \subseteq \{\text{Facebook}, \text{Threads}, \text{Zalo}\}$, danh sách từ khóa/nhóm $\mathcal{Q}$

Ensure: Tập bài viết mới hợp lệ được ghi vào bảng `bai_viet_tho`

for mỗi nền tảng $s \in \mathcal{S}$ **do**
  Nạp cấu hình phiên Cookie_s và tiêu đề giả lập Headers_s từ biến môi trường
  if Cookie_s không hợp lệ **và** s yêu cầu đăng nhập bắt buộc **then**
    Ghi thông báo lỗi phiên vào nhật ký hệ thống và **bỏ qua** s
  end if
  for mỗi mục tiêu tìm kiếm $q \in \mathcal{Q}_s$ **do**
    Gửi yêu cầu `HTTP GET` tới máy chủ nền tảng s với tham số q
    Phân tích HTML/JSON bằng `BeautifulSoup4` để lấy danh sách bài viết $\mathcal{P}$
    for mỗi bài viết $p \in \mathcal{P}$ **do**
      $c \leftarrow \text{NormalizeWhitespace}(p.\text{content})$
      if $\text{Length}(c) \geq 15$ **và không tồn tại** c trong `bai_viet_tho` **then**
        Chèn bản ghi $(c, \text{nguon} = s, \text{trang_thai_ai} = \text{'cho_xu_ly'})$ vào CSDL
      end if
    end for
    Tạm dừng tiến trình trong khoảng ngẫu nhiên $\Delta t \in [2, 5]$ giây
  end for
end for

$$c = 2 \cdot \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right), \quad D = R \cdot c \tag{2.2}$$

Trong đó $\Delta\varphi = \varphi_2 - \varphi_1$ là độ lệch vĩ độ, $\Delta\lambda = \lambda_2 - \lambda_1$ là độ lệch kinh độ (tính theo radian), $R = 6371$ km là bán kính trung bình của Trái Đất, và D là khoảng cách giữa hai điểm tính bằng kilômét (km).

Trên cơ sở khoảng cách không gian Haversine kết hợp ràng buộc khung giờ khởi hành, bài toán điều phối ghép chuyến động (*Dynamic Ride-Sharing*) [17, 13] đánh giá mức độ phù hợp giữa một chuyến xe mở của tài xế t và một yêu cầu tìm xe của hành khách r thông qua hàm mục tiêu tuyến tính có trọng số:

$$\text{Score}(t, r) = 100 \times \left[W_1 \cdot \left(1 - \frac{\Delta D(t,r)}{\Delta D_{\max}}\right) + W_2 \cdot \left(1 - \frac{\Delta T(t,r)}{\Delta T_{\max}}\right) \right] \tag{2.3}$$

Bảng 2.3 giải thích chi tiết ý nghĩa và giá trị thiết lập của các đại lượng toán học trong mô hình định vị và chấm điểm ghép chuyến.

Quy trình tính khoảng cách Haversine và chấm điểm xếp hạng các cặp ghép chuyến khả thi được trình bày chi tiết trong **Giải thuật 3**.

2.2 Công nghệ và thư viện sử dụng

Hệ thống được thiết kế và phát triển theo mô hình kiến trúc phân tầng Client – Server hiện đại, phân tách rõ ràng giữa phân hệ Backend xử lý dữ liệu và trí tuệ nhân tạo với phân hệ Frontend giao diện tương tác người dùng.

2.2.1 Backend (Xử lý dữ liệu và Động cơ AI)

Phân hệ Backend đóng vai trò trung tâm điều phối nghiệp vụ, được xây dựng trên nền tảng ngôn ngữ **Python 3.10+** với các thành phần công nghệ cốt lõi sau:

- **FastAPI Framework & Uvicorn** [10]**:** Xây dựng hệ thống RESTful API hiệu năng cao theo chuẩn bất đồng bộ ASGI, hỗ trợ kiểm tra lược đồ dữ liệu tự động qua Pydantic, nén băng thông GZip và tự động sinh tài liệu kiểm thử trực quan Swagger UI tại đường dẫn `/docs`.
- **Bộ thu thập dữ liệu (*Crawler Modules*)**: Tích hợp ba tiến trình độc lập `crawl_facebook_http.py`, `crawl_zalo_http.py` và `crawl_threads_http.py` sử dụng thư viện `requests` và `BeautifulSoup4` để thu thập và bóc tách bài đăng tự do theo chu kỳ.
- **Hệ quản trị cơ sở dữ liệu (*Database*)** [18]**:** Sử dụng **MySQL 8.0 / TiDB Cloud** cho môi trường vận hành đám mây phân tán và tự động chuyển

Bảng 2.3. Giải thích các đại lượng toán học và ngưỡng ràng buộc trong bài toán định vị – ghép chuyến.

Đại lượng / Tham số	**Ký hiệu**	**Giá trị / Đơn vị**	**Vai trò trong mô hình tính toán**
Tọa độ điểm xuất phát	(Lat_O, Lng_O)	Độ thập phân	Vị trí địa lý của điểm đón hành khách hoặc điểm khởi hành của tài xế.
Tọa độ điểm kết thúc	(Lat_D, Lng_D)	Độ thập phân	Vị trí địa lý của điểm trả hành khách hoặc điểm đích của tài xế.
Độ lệch không gian TB	$\Delta D(t, r)$	$\leq$ 5,0 km	Trung bình cộng khoảng cách Haversine tại hai đầu điểm đi và điểm đến giữa tài xế và khách.
Độ lệch thời gian	$\Delta T(t, r)$	$\leq$ 60 phút	Giá trị tuyệt đối hiệu số thời điểm khởi hành dự kiến giữa hai bên $\lvert\tau_t - \tau_r\rvert$.
Trọng số không gian	W_1	0,60 (60%)	Hệ số ưu tiên mức độ trùng khớp lộ trình địa lý nhằm hạn chế quãng đường đi vòng.
Trọng số thời gian	W_2	0,40 (40%)	Hệ số ưu tiên mức độ trùng khớp khung giờ xuất phát giữa tài xế và hành khách.
Điểm phù hợp tổng hợp	$\text{Score}(t, r)$	$[50\%, 100\%]$	Chỉ số xếp hạng mức độ tối ưu của cặp ghép; các cặp dưới 50% bị loại bỏ.

Giải thuật 3. Tính khoảng cách cầu Haversine và chấm điểm ghép chuyến không gian – thời gian

Require: Chuyến xe tài xế $t(O_t, D_t, \tau_t, q_t)$, yêu cầu hành khách $r(O_r, D_r, \tau_r, p_r)$
Ensure: Điểm phù hợp $\text{Score}(t, r) \in [0, 100]$ hoặc trạng thái `REJECTED`

if $q_t < p_r$ **then**
    return `REJECTED` ▷ Số ghế trống của xe không đủ cho số hành khách
end if
$\Delta T \leftarrow |\tau_t - \tau_r|$ (tính theo phút)
if $\Delta T > 60$ **then**
    return `REJECTED` ▷ Chênh lệch giờ khởi hành vượt ngưỡng 60 phút
end if
$d_{\text{origin}} \leftarrow \text{HaversineDistance}(O_t, O_r)$ ▷ Áp dụng Công thức (2.1)–(2.2)
$d_{\text{dest}} \leftarrow \text{HaversineDistance}(D_t, D_r)$
$\Delta D \leftarrow (d_{\text{origin}} + d_{\text{dest}})/2.0$
if $\Delta D > 5.0$ **then**
    return `REJECTED` ▷ Độ lệch điểm đón/trả vượt bán kính cho phép 5,0 km
end if
$\text{Score} \leftarrow 100 \times \left[0.60 \cdot \left(1 - \frac{\Delta D}{5.0}\right) + 0.40 \cdot \left(1 - \frac{\Delta T}{60.0}\right)\right]$
if $\text{Score} < 50.0$ **then**
    return `REJECTED`
end if
return $\text{Round}(\text{Score}, 2)$

dự phòng về **SQLite** (`xeghep.db`) khi chạy cục bộ, quản lý 5 bảng quan hệ cốt lõi (`users`, `bai_viet_tho`, `chuyen_xe`, `yeu_cau`, `ket_qua_ghep`) đảm bảo tính toàn vẹn tham chiếu.

- **Mô hình AI, Bản đồ & Lập lịch:** Tích hợp API Google Gemini / Groq LLaMA cho mô-đun bóc tách thực thể ngôn ngữ tự nhiên; thư viện Geopy / OpenStreetMap cho dịch vụ định vị tọa độ địa lý; và bộ lập lịch `APScheduler` để tự động chạy giải thuật ghép chuyến định kỳ mỗi 30 phút.

Bảng 2.4 tổng hợp chi tiết ngăn xếp công nghệ, thư viện và vai trò kỹ thuật của từng thành phần trong phân hệ Backend.

Bảng 2.4. Tổng hợp ngăn xếp công nghệ, thư viện và vai trò trong phân hệ Backend.

Nhóm chức năng	Công nghệ / Thư viện	Vai trò và chức năng kỹ thuật trong hệ thống
Nền tảng	`Python 3.10+`	Ngôn ngữ lập trình chính cho toàn bộ dịch vụ API, AI và xử lý dữ liệu.
Web API Server	`FastAPI, Uvicorn`	Cung cấp các điểm cuối RESTful API bất đồng bộ, tài liệu Swagger và nén phản hồi GZip.
Thu thập dữ liệu	`requests, BeautifulSoup4`	Gửi yêu cầu HTTP Session kèm Cookie giả lập trình duyệt và phân tích cấu trúc HTML/JSON.
Động cơ AI / NLP	`Google Gemini, Groq LLaMA`	Phân loại vai trò người đăng và trích xuất thực thể hành trình từ tiếng Việt tự do sang JSON.
Định vị địa lý	`Geopy, OpenStreetMap`	Quy đổi tên địa danh hành chính sang cặp tọa độ (Lat, Lng) phục vụ tính toán Haversine.
Cơ sở dữ liệu	`MySQL 8.0, SQLite`	Lưu trữ bền vững tài khoản, bài viết thô, chuyến xe, yêu cầu và kết quả điều phối ghép chuyến.
Tác vụ lập lịch	`APScheduler`	Tự động kích hoạt giải thuật ghép chuyến định kỳ 30 phút/lần và cập nhật trạng thái hết hạn.

2.2.2 Frontend (Giao diện Dashboard Quản trị và Cổng người dùng)

Phân hệ Frontend phục vụ giao diện giám sát trực tuyến cho quản trị viên và cổng giao dịch cho tài xế, hành khách, được phát triển bằng hệ sinh thái **JavaScript / TypeScript** hiện đại:

- **ReactJS 19 & Vite [1]:** Thư viện xây dựng giao diện theo kiến trúc ứng dụng trang đơn (*Single Page Application – SPA*) kết hợp công cụ đóng gói Vite giúp tối ưu tốc độ tải trang và khả năng hiển thị mượt mà trên cả máy tính lẫn thiết bị di động.
- **Tailwind CSS 4 & React Router 7:** Hệ thống thiết kế giao diện tiện ích (*utility-first*) giúp đồng bộ hóa bảng màu, biểu đồ thống kê và điều hướng phân quyền chặt chẽ giữa Quản trị viên (`/dashboard`), Tài xế (`/driver`) và Hành khách (`/customer`).
- **Trực quan hóa dữ liệu và Giao tiếp API:** Sử dụng các bảng điều khiển tương tác (`Dashboard.jsx`, `DriverDashboard.jsx`, `CustomerDashboard.jsx`) kết hợp thư viện giao tiếp HTTP tập trung (`api.js`) và bản đồ trực quan để hiển thị tỷ trọng nguồn mạng xã hội, tuyến đường phổ biến và điểm phù hợp của từng cặp ghép chuyến.

Bảng 2.5 mô tả chi tiết cấu trúc các thành phần giao diện và công nghệ sử dụng trong phân hệ Frontend.

2.3 Các công trình liên quan và Tính sáng tạo của đề tài

Trên thị trường vận tải công nghệ tại Việt Nam hiện nay đã có một số nền tảng chia sẻ chuyến đi thương mại phổ biến như GrabShare, BeShare hay sàn đặt xe đường dài Đi Chung. Bên cạnh đó, trong lĩnh vực nghiên cứu học thuật, các công trình của Agatz và cộng sự [17], Furuhata và cộng sự [13] hay Alonso-Mora và cộng sự [15] đã đề xuất nhiều mô hình tối ưu ghép xe theo thời gian thực trên đồ thị giao thông. Tuy nhiên, giữa các giải pháp hiện có và đề xuất của đồ án tồn tại sự khác biệt căn bản về cách tiếp cận nguồn dữ liệu đầu vào và mô hình vận hành.

Bảng 2.6 đối sánh chi tiết các đặc tính kỹ thuật và mô hình nghiệp vụ giữa các nền tảng gọi xe truyền thống với hệ thống đề xuất của đề tài.

Bảng 2.5. Đặc tả các thành phần giao diện và thư viện sử dụng trong phân hệ Frontend.

Thành phần	**Tệp mã nguồn / Thư viện**	**Chức năng hiển thị và tương tác người dùng**
Lõi giao diện SPA	`React 19, Vite 8`	Quản lý trạng thái thành phần, vòng đời dữ liệu và đóng gói bản build tĩnh tối ưu.
Định tuyến phân quyền	`React Router 7 (App.jsx)`	Điều hướng giữa trang chủ, đăng nhập/đăng ký và các phân vùng chức năng theo vai trò.
Bảng điều khiển Admin	`Dashboard.jsx`	Giám sát chỉ số KPI, bật/tắt 3 bot thu thập dữ liệu, kích hoạt AI và quản lý các bảng dữ liệu.
Cổng Tài xế & Khách	`DriverDashboard.jsx, CustomerDashboard.jsx`	Hỗ trợ đăng chuyến xe mở, đăng yêu cầu tìm xe, tra cứu cặp ghép đề xuất và đánh giá sao.
Tầng giao tiếp API	`api.js, Fetch API`	Gửi yêu cầu RESTful tới Backend kèm cơ chế tự động thử lại (*retry*) và xử lý lỗi mạng.

Đóng góp sáng tạo của đề tài: Điểm mới và giá trị thực tiễn vượt trội của đồ án chính là việc thiết kế một đường ống xử lý tự động khép kín (*End-to-End Automated Pipeline*), kết hợp hài hòa giữa kỹ thuật thu thập dữ liệu đa kênh nhẹ bằng HTTP Session, trí tuệ nhân tạo thế hệ mới (LLM) để đọc hiểu ngữ nghĩa tiếng Việt tự do, và giải thuật tối ưu hóa không gian – thời gian Haversine. Giải pháp này giúp khai thác hiệu quả nguồn dữ liệu cộng đồng khổng lồ, biến động theo thời gian thực trên các nền tảng mạng xã hội — nơi diễn ra nhu cầu ghép xe liên tỉnh tự nhiên lớn nhất hiện nay mà các ứng dụng gọi xe truyền thống chưa thể tiếp cận tự động.

Bảng 2.6. Bảng so sánh đối sánh giữa các nền tảng gọi xe truyền thống và Đề tài.

Tiêu chí so sánh	**Ứng dụng truyền thống (Grab, Be, Đi Chung)**	**Hệ thống Xe Ghép AI (Đề tài)**
Nguồn dữ liệu	Phụ thuộc hoàn toàn vào dữ liệu người dùng nhập thủ công trên ứng dụng chuyên biệt.	Tự động thu thập và "lắng nghe" dữ liệu tự do đa kênh từ mạng xã hội (Facebook, Threads, Zalo) kết hợp cổng Web.
Xử lý ngôn ngữ	Không hỗ trợ đọc hiểu văn bản tự do; bắt buộc chọn địa danh cố định từ biểu mẫu.	Tự động bóc tách bài đăng tự do, từ viết tắt địa phương và tiếng lóng bằng Mô hình ngôn ngữ lớn (LLM).
Thói quen người dùng	Yêu cầu cả tài xế và hành khách tải ứng dụng mới, thao tác đăng ký phức tạp.	Tận dụng trực tiếp thói quen đăng bài tự do sẵn có của cộng đồng trên các hội nhóm mạng xã hội.
Cơ chế điều phối	Định giá cố định qua tổng đài trung gian, thu chiết khấu hoa hồng cao trên từng chuyến đi.	Tự động lọc ràng buộc không gian – thời gian, chấm điểm độ phù hợp và kết nối trực tiếp hai bên tối ưu chi phí.
Phân tích dữ liệu	Chỉ thống kê được giao dịch phát sinh nội bộ bên trong hệ sinh thái ứng dụng.	Tổng hợp và phân tích được xu hướng cung – cầu tuyến đường thực tế từ cộng đồng mạng xã hội.

3

Phân tích và thiết kế hệ thống

Mục tiêu của chương: *Trình bày chi tiết quá trình phân tích bài toán thực tế và đặc tả các yêu cầu chức năng, phi chức năng của hệ thống. Từ đó, chương này xây dựng bộ thiết kế kiến trúc hoàn chỉnh bao gồm: sơ đồ phân rã chức năng, sơ đồ luồng dữ liệu tổng thể, sơ đồ ca sử dụng (Use Case), sơ đồ trình tự (Sequence Diagram), thiết kế cơ sở dữ liệu quan hệ (ERD) cùng mô hình trích xuất AI và giải thuật điều phối ghép chuyến tối ưu.*

3.1 Phân tích bài toán và yêu cầu hệ thống

3.1.1 Phân tích bài toán thực tế

Hiện nay, nhu cầu đi chung xe, ghép chuyến (*carpooling*) trên các tuyến đường liên tỉnh và tuyến kết nối sân bay là rất lớn. Người dùng (bao gồm cả tài xế có ghế trống và hành khách cần tìm xe) thường xuyên đăng tải thông tin trực tiếp lên các hội nhóm cộng đồng trên mạng xã hội như Facebook, Zalo và Threads. Tuy nhiên, luồng dữ liệu này hoàn toàn phi cấu trúc, phân tán trên nhiều nền tảng, bài đăng trôi nhanh theo thời gian và sử dụng nhiều từ viết tắt địa phương, tiếng lóng giao thông.

Việc tìm kiếm và đối chiếu lịch trình thủ công khiến cả hành khách lẫn tài xế mất nhiều thời gian theo dõi, dễ bỏ lỡ chuyến đi phù hợp và khó đánh giá mức độ trùng khớp về lộ trình địa lý cũng như khung giờ khởi hành. Do đó, bài toán đặt ra là xây dựng một nền tảng thông minh có khả năng tự động hóa khép kín từ khâu thu thập dữ liệu mạng xã hội đa kênh, đọc hiểu ngôn ngữ tự nhiên tiếng Việt để cấu trúc hóa thông tin hành trình, đến việc định vị tọa độ không gian và đề xuất tự động các cặp ghép chuyến có độ phù hợp cao nhất.

3.1.2 Phân tích yêu cầu chức năng cụ thể

Dựa trên quy trình nghiệp vụ thực tế và các nhóm đối tượng sử dụng, hệ thống được phân chia thành bốn nhóm yêu cầu chức năng cốt lõi sau:

- **Nhóm chức năng 1 – Quản lý người dùng và phân quyền truy cập:** Hỗ trợ đăng ký tài khoản mới, đăng nhập xác thực phiên làm việc, phân tách vai trò rõ ràng giữa *Hành khách* (`khach_hang`), *Tài xế* (`tai_xe`) và *Quản trị viên* (`admin`), đồng thời quản lý hồ sơ cá nhân và lịch sử giao dịch.
- **Nhóm chức năng 2 – Thu thập và quản lý dữ liệu đa nguồn:** Tự động thu thập bài viết công khai từ Facebook, Zalo và Threads theo chu kỳ định kỳ thông qua các tiến trình *Crawler* chạy ngầm; kiểm tra khử trùng lặp nội dung; lưu trữ vào kho bài viết thô (`bai_viet_tho`) và cho phép quản trị viên bật/tắt hoặc kích hoạt quét thủ công từng kênh.
- **Nhóm chức năng 3 – Trích xuất thực thể AI (NLP/LLM) và định vị địa lý:** Tự động phân loại ý định người đăng (tài xế tìm khách hay hành khách tìm xe), bóc tách 6 thực thể cốt lõi (điểm đi, điểm đến, thời gian khởi hành, số ghế/số khách, số điện thoại liên hệ) sang chuẩn JSON thông qua Mô hình ngôn ngữ lớn (Google Gemini, Groq LLaMA) kết hợp bộ lọc Regex dự phòng, đồng thời quy đổi địa danh thành tọa độ địa lý (Lat, Lng) bằng dịch vụ Geocoding.
- **Nhóm chức năng 4 – Điều phối ghép chuyến thông minh và hiển thị Dashboard:** Cho phép tài xế đăng chuyến xe mở, hành khách đăng yêu cầu tìm xe; tự động kích hoạt giải thuật lọc ràng buộc không gian ($\Delta D \leq 5{,}0$ km) và thời gian ($\Delta T \leq 60$ phút), chấm điểm độ phù hợp (`Score`) để đề xuất danh sách ghép nối tối ưu trên giao diện Web Dashboard cho từng vai trò.

3.1.3 Phân tích yêu cầu phi chức năng

Bên cạnh các chức năng nghiệp vụ, hệ thống phải đáp ứng nghiêm ngặt các tiêu chí kỹ thuật về hiệu năng, độ chính xác và tính sẵn sàng:

- **Độ chính xác nhận diện ngôn ngữ tiếng Việt:** Mô-đun NLP/LLM phải hiểu được các từ viết tắt địa danh, tiếng lóng vận tải và cấu trúc câu tự do, đạt chỉ số F_1-Score tối thiểu 80%.
- **Hiệu năng và tốc độ phản hồi:** Thời gian xử lý trích xuất AI trung bình dưới 5 giây cho mỗi bài viết; thời gian chạy giải thuật so khớp không gian – thời gian dưới 1 giây cho mỗi lượt điều phối.

- **Khả năng chịu lỗi và mở rộng:** Hỗ trợ cơ chế xoay vòng nhiều khóa API (*Round-Robin API Key Rotation*) và bộ phân tích Regex dự phòng khi dịch vụ LLM bên ngoài bị giới hạn tần suất; cơ sở dữ liệu quan hệ hỗ trợ vận hành linh hoạt trên cả MySQL đám mây và SQLite cục bộ.

Toàn bộ các tiêu chí yêu cầu kỹ thuật của hệ thống được tổng hợp chi tiết trong **Bảng 3.1**.

Bảng 3.1. Thống kê tiêu chí phân tích yêu cầu hệ thống điều phối xe ghép.

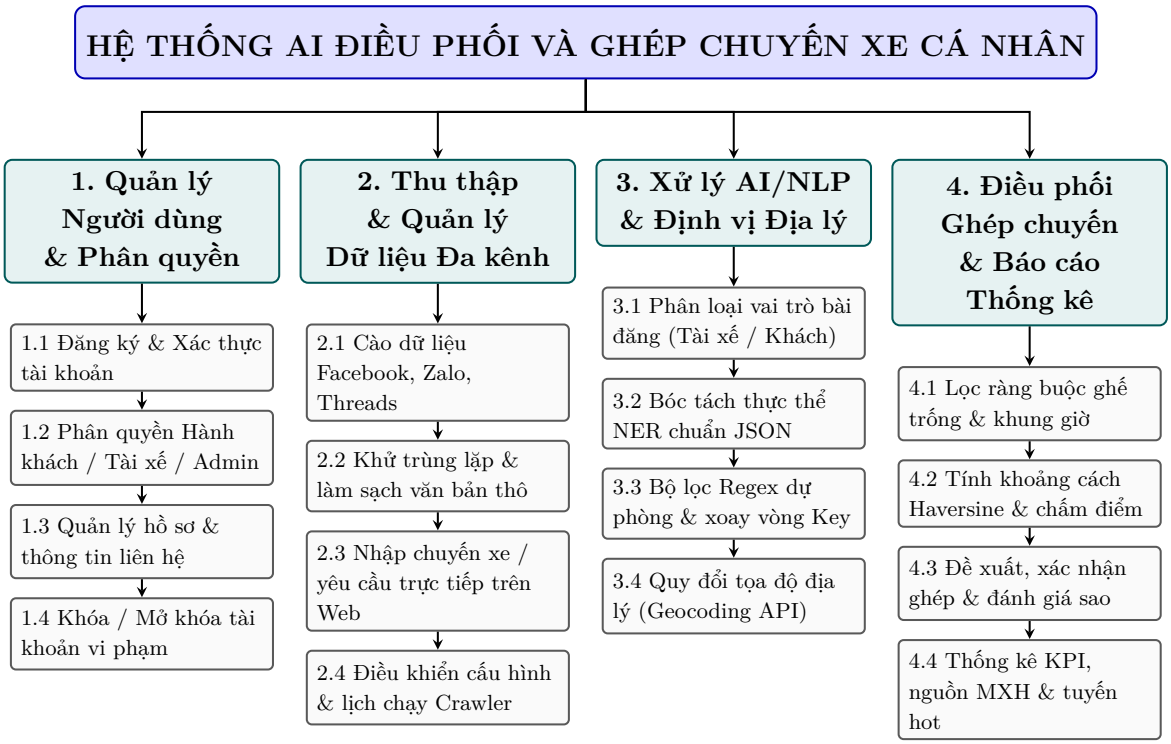
Nhóm yêu cầu	Tên yêu cầu	Mô tả tiêu chí chi tiết
Chức năng	**Quản lý phân quyền**	Đăng ký, đăng nhập và phân tách giao diện theo 3 vai trò: Hành khách, Tài xế và Quản trị viên.
Chức năng	**Thu thập đa kênh**	Định kỳ thu thập bài viết thô từ Facebook, Zalo, Threads và khử trùng lặp trước khi lưu CSDL.
Chức năng	**Bóc tách AI (NER)**	Phân loại vai trò và trích xuất chuẩn JSON gồm Điểm đi, Điểm đến, Thời gian, Số lượng, SĐT.
Chức năng	**Điều phối ghép xe**	Quy đổi tọa độ Geocoding, lọc độ lệch lộ trình ($\Delta D \leq 5{,}0$ km) và thời gian ($\Delta T \leq 60$ phút).
Phi chức năng	**Tốc độ xử lý**	Phản hồi trích xuất AI < 5 giây/bài đăng; thực thi giải thuật ghép chuyến < 1 giây/lượt.
Phi chức năng	**Độ chính xác**	Chỉ số F_1-Score trích xuất thực thể hành trình tiếng Việt đạt tối thiểu 80%.
Phi chức năng	**Tính sẵn sàng**	Tự động xoay vòng khóa API LLM, có bộ lọc Regex dự phòng và lập lịch chạy ngầm 24/7.

3.2 Thiết kế kiến trúc và luồng xử lý hệ thống

3.2.1 Sơ đồ phân rã chức năng (Functional Decomposition Diagram)

Để tổ chức mã nguồn theo hướng mô-đun hóa rõ ràng, toàn bộ nghiệp vụ của hệ thống **AI Điều phối và Ghép chuyến Xe cá nhân** được phân rã thành 4 phân

hệ chức năng chính và 16 chức năng thành phần như minh họa tại **Hình 3.1**.



Hình 3.1. Sơ đồ phân rã chức năng của hệ thống AI điều phối và ghép chuyến xe cá nhân.

3.2.2 Sơ đồ luồng xử lý tổng thể hệ thống (End-to-End System Flow)

Luồng vận hành của hệ thống được thiết kế theo kiến trúc đường ống xử lý tự động khép kín (*End-to-End Automated Pipeline*) gồm 5 tầng liên thông từ khâu thu thập dữ liệu thô đến khi trả kết quả ghép chuyến tối ưu cho người dùng, như minh họa trong **Hình 3.2**.

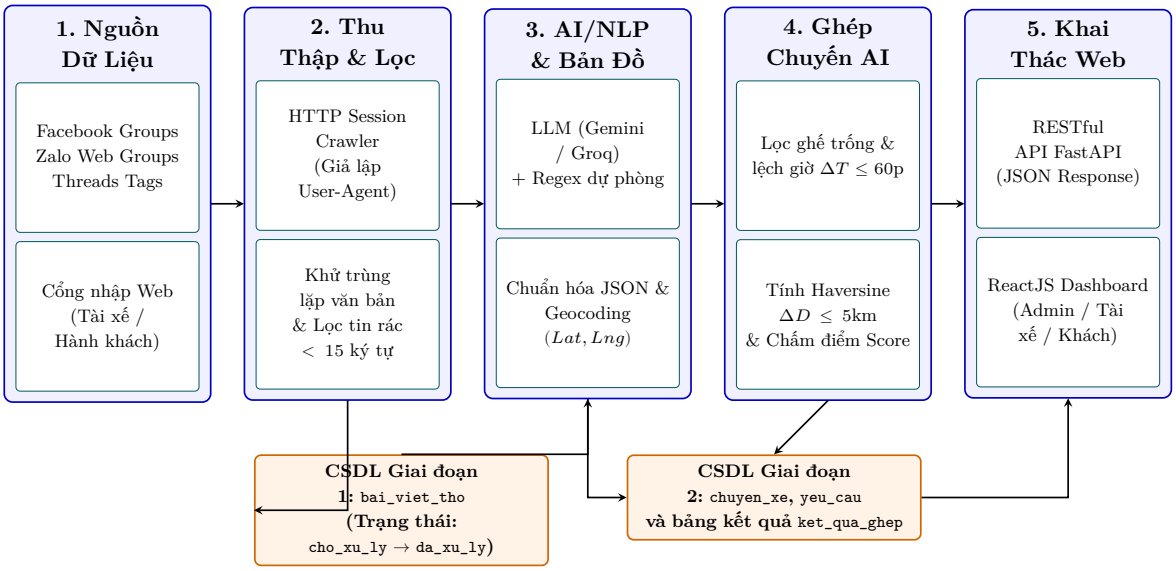
Mô tả chi tiết từng bước trong luồng xử lý tổng thể:

1. **Bước 1 – Tiếp nhận nguồn dữ liệu đa kênh:** Dữ liệu đi vào hệ thống từ hai luồng song song: (a) các bài đăng tự do công khai trên hội nhóm Facebook, Zalo, Threads và (b) dữ liệu người dùng nhập trực tiếp thông qua giao diện cổng Web.

2. **Bước 2 – Thu thập, làm sạch và chống trùng lặp:** Các bộ thu thập HTTP Crawler gửi yêu cầu truy vấn kèm phiên đăng nhập, loại bỏ các chuỗi quảng cáo quá ngắn (< 15 ký tự) hoặc không chứa số điện thoại, đối chiếu nội dung với cơ sở dữ liệu để tránh lưu trùng, sau đó ghi bản ghi mới vào bảng `bai_viet_tho` với trạng thái `cho_xu_ly`.

Bảng 3.2. Bảng đặc tả chi tiết các chức năng cụ thể của hệ thống theo từng phân hệ.

Mã	Tên chức năng	Tác nhân	Mô tả xử lý nghiệp vụ cụ thể
CN01	**Đăng ký & Xác thực**	Khách, Tài xế, Admin	Kiểm tra tính duy nhất của tên đăng nhập/SĐT, băm mật khẩu an toàn và cấp phiên truy cập theo vai trò.
CN02	**Thu thập dữ liệu MXH**	Hệ thống, Admin	Chạy 3 mô-đun HTTP Crawler (Facebook, Zalo, Threads) định kỳ hoặc theo lệnh từ Dashboard để lấy bài viết mới.
CN03	**Đăng tin trực tiếp**	Khách, Tài xế	Cho phép tài xế tạo chuyến xe mở hoặc hành khách tạo yêu cầu tìm xe trực tiếp trên cổng Web (dạng biểu mẫu hoặc văn bản tự do).
CN04	**Trích xuất AI (NLP)**	Hệ thống, Admin	Gọi API Gemini/Groq kèm Prompt định hướng để bóc tách JSON; tự động chuyển sang Regex nếu quá tải API.
CN05	**Định vị Geocoding**	Hệ thống	Chuyển đổi địa danh điểm đi và điểm đến sang cặp tọa độ (Lat, Lng) thông qua dịch vụ OpenStreetMap/Geopy.
CN06	**Điều phối ghép chuyến**	Hệ thống, Admin	So khớp từng cặp chuyến xe – yêu cầu thỏa mãn $\Delta D \leq 5{,}0$ km, $\Delta T \leq 60$ phút và lưu điểm xếp hạng vào `ket_qua_ghep`.
CN07	**Xác nhận & Đánh giá**	Khách, Tài xế	Xem danh sách đối tác phù hợp do AI đề xuất, liên hệ trực tiếp qua SĐT, chốt chuyến và gửi đánh giá chất lượng sau chuyến đi.
CN08	**Quản trị & Thống kê**	Admin	Theo dõi biểu đồ tỷ trọng nguồn dữ liệu, hiệu suất bóc tách AI, tuyến đường phổ biến và quản lý toàn bộ dữ liệu hệ thống.



Hình 3.2. Sơ đồ luồng xử lý dữ liệu tổng thể từ thu thập đa kênh đến điều phối ghép chuyến.

3. **Bước 3 – Phân tích ngữ nghĩa bằng AI/NLP:** Tiến trình xử lý AI đọc các bài viết ở trạng thái `cho_xu_ly`, gửi tới Mô hình ngôn ngữ lớn (Gemini/Groq) để phân loại vai trò (`tai_xe` hoặc `khach_hang`) và trích xuất 6 thực thể sang định dạng JSON. Nếu API gặp sự cố giới hạn tần suất, hệ thống tự động xoay vòng khóa API hoặc kích hoạt bộ phân tích biểu thức chính quy (Regex) dự phòng.

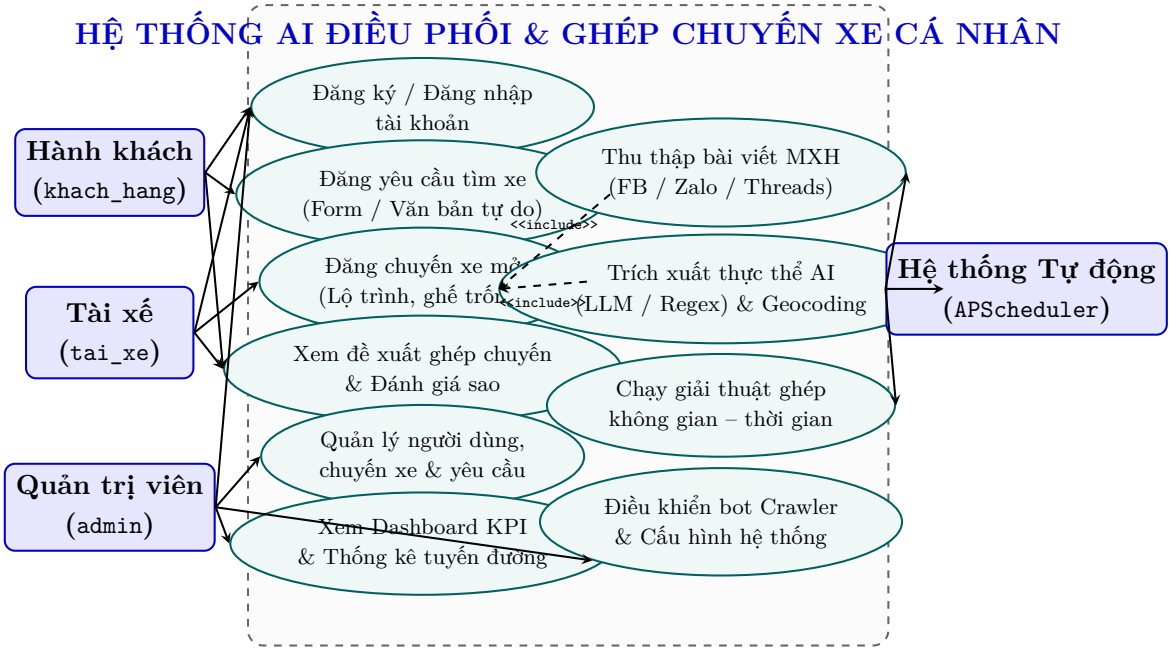
4. **Bước 4 – Chuẩn hóa địa danh và định vị tọa độ (Geocoding):** Từ tên điểm đi và điểm đến đã được chuẩn hóa cấp hành chính, hệ thống gọi dịch vụ bản đồ OpenStreetMap/Geopy để lấy cặp tọa độ vĩ độ – kinh độ (Lat, Lng) thực tế, sau đó lưu bản ghi hoàn chỉnh vào bảng `chuyen_xe` (nếu là tài xế) hoặc `yeu_cau` (nếu là hành khách) và cập nhật bài viết thô sang trạng thái `da_xu_ly`.

5. **Bước 5 – Lọc ràng buộc và chấm điểm ghép chuyến:** Định kỳ mỗi 30 phút (hoặc ngay khi có chuyến mới), bộ lập lịch `APScheduler` kích hoạt giải thuật điều phối: kiểm tra điều kiện đủ ghế trống, lọc chênh lệch thời gian khởi hành $\Delta T \leq 60$ phút, tính khoảng cách cầu Haversine tại điểm đón và điểm trả sao cho độ lệch trung bình $\Delta D \leq 5{,}0$ km, sau đó tính điểm phù hợp Score $\in [50\%, 100\%]$ và lưu vào bảng `ket_qua_ghep`.

6. **Bước 6 – Hiển thị kết quả và kết nối giao dịch:** Phân hệ Frontend ReactJS gọi các điểm cuối RESTful API của FastAPI để hiển thị danh sách chuyến đi/hành khách phù hợp nhất theo thứ tự điểm số giảm dần, giúp hai bên chủ động liên hệ chốt chuyến và quản trị viên giám sát toàn diện hiệu năng hệ thống.

3.2.3 Sơ đồ ca sử dụng (Use Case Diagram)

Để phản ánh chính xác mô hình vận hành đa vai trò của ứng dụng, **Hình 3.3** mô tả mối quan hệ tương tác giữa bốn tác nhân chính (*Hành khách*, *Tài xế*, *Quản trị viên* và *Hệ thống tự động*) với các ca sử dụng cốt lõi bên trong ranh giới hệ thống.



Hình 3.3. Sơ đồ Use Case tổng quan thể hiện tương tác của 4 tác nhân chính với hệ thống.

Mô tả chi tiết các tác nhân và ca sử dụng trong sơ đồ Use Case:

- **Tác nhân Hành khách** (`khach_hang`)**:** Thực hiện đăng ký, đăng nhập tài khoản; khởi tạo yêu cầu tìm xe thông qua biểu mẫu có cấu trúc hoặc dán trực tiếp đoạn văn bản tự do (hệ thống tự động gọi mô-đun AI/Geocoding để chuẩn hóa); theo dõi danh sách các chuyến xe phù hợp nhất do AI đề xuất, liên hệ tài xế và gửi đánh giá chất lượng sau chuyến đi.

- **Tác nhân Tài xế** (`tai_xe`)**:** Đăng nhập vào phân vùng quản lý của tài xế; đăng tải thông tin chuyến xe mở (điểm xuất phát, điểm đích, giờ khởi hành, số ghế trống); tra cứu danh sách hành khách tiện chuyến dọc lộ trình và xác nhận nhận khách.

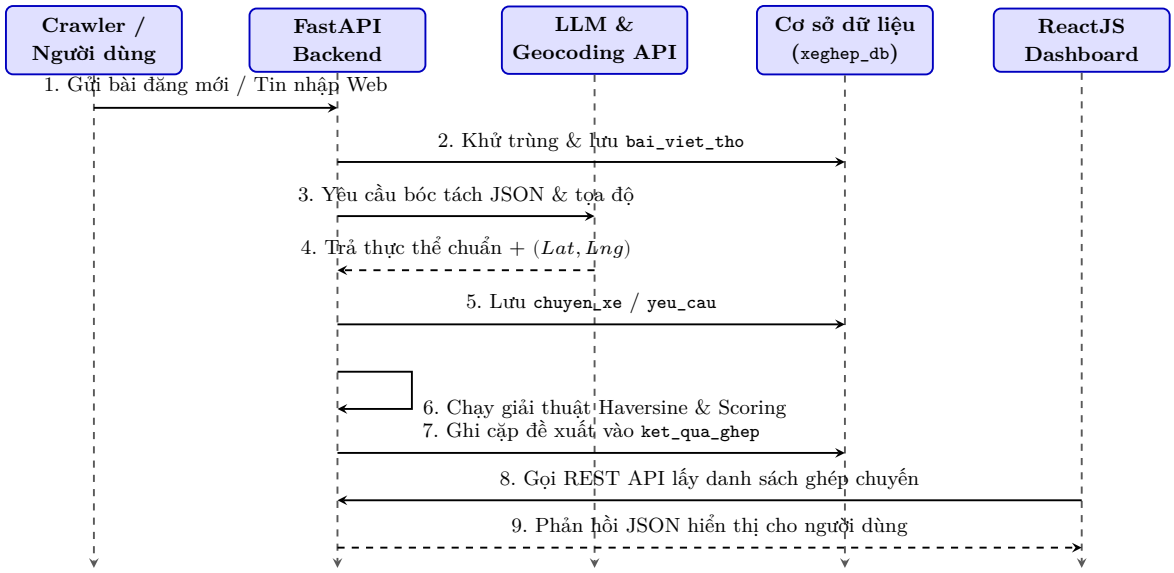
- **Tác nhân Quản trị viên** (`admin`)**:** Toàn quyền giám sát bảng điều khiển tổng hợp (Dashboard KPI, biểu đồ tỷ trọng nguồn mạng xã hội, thống kê tuyến đường phổ biến); điều khiển bật/tắt hoặc kích hoạt thủ công các tiến trình thu thập dữ liệu, trích xuất AI; quản lý, chỉnh sửa hoặc xóa các bản ghi người dùng, bài viết thô, chuyến xe và yêu cầu.

- **Tác nhân Hệ thống tự động** (`APScheduler`)**:** Hoạt động chạy ngầm định

kỳ trên máy chủ Backend để thực thi 3 ca sử dụng tự động: (1) Quét bài viết mới từ Facebook, Zalo, Threads; (2) Gọi mô-đun AI/NLP và Geocoding để chuyển đổi văn bản thô thành tọa độ hành trình; (3) Thực thi giải thuật so khớp Haversine để sinh các cặp đề xuất ghép chuyến mới.

3.2.4 Sơ đồ trình tự (Sequence Diagram)

Hình 3.4 biểu diễn trình tự trao đổi thông điệp theo trục thời gian giữa 5 thành phần kiến trúc chính: *Crawler / Người dùng*, *FastAPI Backend*, *LLM & Geocoding API*, *Cơ sở dữ liệu (CSDL)* và *ReactJS Dashboard*.



Hình 3.4. Sơ đồ trình tự biểu diễn luồng xử lý dữ liệu từ lúc thu thập đến khi hiển thị kết quả.

Mô tả chi tiết trình tự thực thi (Bước 1 – Bước 9):

1. **Bước 1:** Bộ thu thập dữ liệu (*Crawler*) gửi các bài đăng mới thu thập từ mạng xã hội (hoặc người dùng gửi bài đăng trực tiếp từ giao diện Web) tới máy chủ *FastAPI Backend*.

2. **Bước 2:** *FastAPI Backend* kiểm tra điều kiện lọc nhiễu, chống trùng lặp và lưu văn bản gốc vào bảng `bai_viet_tho` trong *Cơ sở dữ liệu* với trạng thái `cho_xu_ly`.

3. **Bước 3:** *FastAPI Backend* đóng gói nội dung văn bản vào câu lệnh Prompt chuyên biệt và gửi yêu cầu tới dịch vụ *LLM & Geocoding API*.

4. **Bước 4:** Dịch vụ AI và bản đồ phân tích ngữ nghĩa tiếng Việt, trả về đối tượng JSON chứa vai trò, điểm đi, điểm đến, thời gian, số ghế/khách, số điện thoại cùng cặp tọa độ địa lý (Lat, Lng).

5. **Bước 5:** Dựa vào vai trò đã phân loại, Backend chèn dữ liệu có cấu trúc vào bảng `chuyen_xe` hoặc bảng `yeu_cau`, đồng thời cập nhật trạng thái bài viết thô thành `da_xu_ly`.

6. **Bước 6:** Bộ lập lịch hoặc hàm kích hoạt nội bộ trên *FastAPI Backend* thực thi giải thuật lọc ràng buộc thời gian, tính khoảng cách Haversine và chấm điểm độ phù hợp giữa các chuyến xe và yêu cầu đang mở.

7. **Bước 7:** Các cặp ghép chuyến đạt ngưỡng điểm $\geq 50\%$ được ghi nhận vào bảng trung gian `ket_qua_ghep`.

8. **Bước 8 & Bước 9:** Khi quản trị viên, tài xế hoặc hành khách truy cập *ReactJS Dashboard*, giao diện gửi yêu cầu HTTP GET tới Backend và nhận về danh sách kết quả định dạng JSON để hiển thị trực quan.

3.3 Thiết kế cơ sở dữ liệu

3.3.1 Sơ đồ thực thể – mối quan hệ (ERD)

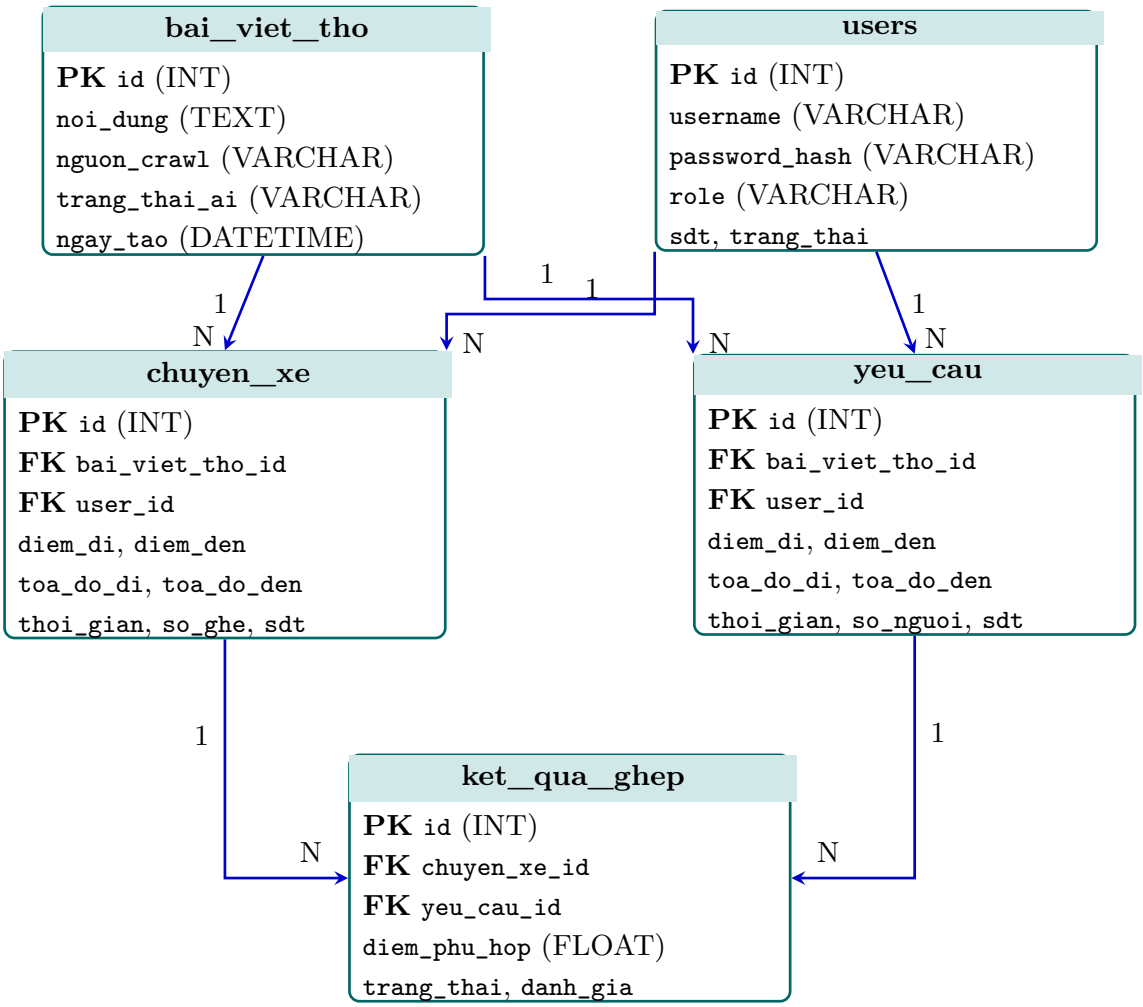
Cơ sở dữ liệu quan hệ của hệ thống (`xeghep_db`) được thiết kế gồm 5 bảng thực thể chính nhằm đảm bảo tính toàn vẹn dữ liệu, tránh dư thừa và hỗ trợ truy vấn ghép nối hiệu năng cao, như minh họa tại **Hình 3.5**.

Mô tả chi tiết các mối quan hệ trong sơ đồ ERD:

- **Quan hệ giữa `bai_viet_tho` với `chuyen_xe` và `yeu_cau` (1 – N):** Một bài đăng thô thu thập từ mạng xã hội có thể chứa nhiều lịch trình di chuyển khác nhau (ví dụ tài xế đăng cả chiều đi và chiều về). Khi mô-đun AI phân tách, một bản ghi trong `bai_viet_tho` có thể sinh ra 1 hoặc nhiều bản ghi trong `chuyen_xe` hoặc `yeu_cau`.

- **Quan hệ giữa `users` với `chuyen_xe` và `yeu_cau` (1 – N):** Mỗi tài khoản người dùng đã đăng ký trên cổng Web (tài xế hoặc hành khách) có thể khởi tạo nhiều chuyến xe mở hoặc nhiều yêu cầu tìm xe theo thời gian. Đối với dữ liệu thu thập tự động từ mạng xã hội, trường khóa ngoại `user_id` nhận giá trị `NULL`.

- **Quan hệ nhiều – nhiều (N – N) thông qua thực thể trung gian `ket_qua_ghep`:** Một chuyến xe của tài xế có thể phù hợp với nhiều yêu cầu tìm xe của các hành khách khác nhau, và ngược lại một hành khách cũng có thể nhận được nhiều đề xuất chuyến xe. Bảng `ket_qua_ghep` lưu trữ từng cặp khóa ngoại `chuyen_xe_id` và `yeu_cau_id` kèm theo chỉ số `diem_phu_hop` và trạng thái xác nhận giao dịch.



Hình 3.5. Sơ đồ thực thể – mối quan hệ (ERD) của cơ sở dữ liệu `xeghep_db`.

3.3.2 Thiết kế chi tiết các bảng dữ liệu

Dựa trên sơ đồ ERD, cấu trúc vật lý của 5 bảng dữ liệu trong cơ sở dữ liệu quan hệ `xeghep_db` được đặc tả chi tiết trong **Bảng 3.3**.

3.4 Thiết kế mô hình AI và giải thuật điều phối

3.4.1 Thiết kế mô hình NLP/LLM trích xuất thông tin hành trình

Mô-đun trích xuất thông tin hành trình được xây dựng theo kiến trúc lai (*Hybrid Extraction Architecture*), kết hợp năng lực suy luận ngữ cảnh của Mô hình ngôn ngữ lớn (Google Gemini / Groq LLaMA [11, 9]) với bộ lọc biểu thức chính quy (Regex) dự phòng:

- **Đầu vào (*Input*):** Chuỗi văn bản thô từ trường `noi_dung` của bảng `bai_viet_tho` sau khi đã được loại bỏ các ký tự điều khiển rác và khoảng trắng dư thừa.
- **Thiết kế câu lệnh định hướng (*Prompt Engineering*):** Câu lệnh hệ thống chỉ định rõ vai trò chuyên gia phân tích dữ liệu vận tải tiếng Việt, cung cấp từ điển ánh xạ các từ viết tắt phổ biến (như *HN* → *Hà Nội*, *ĐN* → *Đà Nẵng*, *NHS* → *Ngũ Hành Sơn*, *mống* → *hành khách*) kèm theo các ví dụ mẫu (*Few-shot Examples*).
- **Ràng buộc đầu ra (*Structured JSON Output*):** Mô hình bắt buộc trả về một mảng đối tượng JSON chuẩn gồm 6 khóa: `vai_tro`, `diem_di`, `diem_den`, `thoi_gian`, `so_luong` và `sdt`, giúp loại bỏ hoàn toàn văn bản giải thích dư thừa và sẵn sàng nạp vào dịch vụ Geocoding.

3.4.2 Thiết kế giải thuật điều phối ghép chuyến tối ưu

Quá trình so khớp và đề xuất ghép chuyến tự động được thiết kế theo quy trình lọc đa tầng gồm 3 bước nhằm tối ưu hóa độ chính xác không gian – thời gian và giảm thiểu độ phức tạp tính toán:

1. **Bước 1 – Lọc ràng buộc nghiệp vụ và thời gian (*Heuristic Filtering*):** Hệ thống chỉ xét các cặp (t, r) giữa một chuyến xe mở t của tài xế và một yêu cầu tìm xe r của hành khách thỏa mãn đồng thời hai điều kiện tiên quyết: (a) số ghế trống của xe phải lớn hơn hoặc bằng số lượng hành khách ($q_t \geq p_r$), và (b) độ chênh lệch thời gian khởi hành dự kiến không vượt quá ngưỡng cho phép

Bảng 3.3. Mô tả thuộc tính chi tiết các bảng trong cơ sở dữ liệu `xeghep_db`.

Tên bảng	Tên trường	Kiểu dữ liệu	Mô tả / Ràng buộc toàn vẹn
`users`	`id, username`	INT, VARCHAR	Khóa chính (PK) tự tăng; tên đăng nhập duy nhất (`UNIQUE`).
`users`	`password_hash, role`	VARCHAR(255)	Mật khẩu đã băm và vai trò phân quyền (`admin`, `tai_xe`, `khach_hang`).
`bai_viet_tho`	`id, noi_dung`	INT, TEXT	Khóa chính (PK); nội dung văn bản gốc thu thập từ mạng xã hội hoặc cổng Web.
`bai_viet_tho`	`nguon_crawl, trang_thai_ai`	VARCHAR(50)	Nguồn nền tảng (`facebook`, `zalo`, `threads`, `web`) và trạng thái xử lý AI.
`chuyen_xe`	`id, bai_viet_tho_id`	INT, INT	Khóa chính (PK); Khóa ngoại (FK) liên kết tới bài viết thô nguồn.
`chuyen_xe`	`diem_di, diem_den`	VARCHAR(255)	Tên địa danh xuất phát và địa danh đích đã được AI chuẩn hóa hành chính.
`chuyen_xe`	`toa_do_di, toa_do_den`	VARCHAR(100)	Cặp tọa độ địa lý (Lat, Lng) quy đổi từ dịch vụ Geocoding phục vụ tính Haversine.
`chuyen_xe`	`thoi_gian, so_ghe, sdt`	VARCHAR, INT	Thời điểm khởi hành dự kiến, số ghế trống hiện có và số điện thoại tài xế.
`yeu_cau`	`id, so_nguoi, sdt`	INT, INT, VARCHAR	Khóa chính (PK); số lượng hành khách cần đi và số điện thoại liên hệ của khách.
`ket_qua_ghep`	`chuyen_xe_id, yeu_cau_id`	INT, INT	Cặp khóa ngoại (FK) tham chiếu tới chuyến xe của tài xế và yêu cầu của hành khách.
`ket_qua_ghep`	`diem_phu_hop, trang_thai`	FLOAT, VARCHAR	Điểm đánh giá mức độ tương đồng $(50\% - 100\%)$ và trạng thái chốt chuyến.

($\Delta T = |\tau_t - \tau_r| \leq 60$ phút).

2. **Bước 2 – Tính toán độ lệch không gian (*Geospatial Analysis*):** Từ cặp tọa độ (Lat, Lng) của điểm xuất phát và điểm kết thúc đã được Geocoding, hệ thống áp dụng công thức cầu Haversine [12] để tính khoảng cách đường chim bay tại điểm đón d_{origin} và điểm trả d_{dest}. Độ lệch không gian trung bình $\Delta D = \frac{d_{\text{origin}} + d_{\text{dest}}}{2}$ phải thỏa mãn bán kính đón/trả tối đa $\Delta D \leq 5{,}0$ km.

3. **Bước 3 – Chấm điểm tổng hợp và xếp hạng (*Scoring & Ranking*):** Các cặp vượt qua hai tầng lọc trên được tính điểm phù hợp theo hàm mục tiêu tuyến tính có trọng số [17] với các tham số cấu hình quy định tại **Bảng 3.4**:

$$\text{Score}(t, r) = 100 \times \left[W_1 \cdot \left(1 - \frac{\Delta D}{\text{Max_Distance}}\right) + W_2 \cdot \left(1 - \frac{\Delta T}{\text{Max_Time}}\right) \right] \tag{3.1}$$

Các cặp ghép đạt điểm $\text{Score}(t, r) \geq 50\%$ được chèn vào bảng `ket_qua_ghep` và sắp xếp theo thứ tự điểm số giảm dần trên giao diện người dùng.

Bảng 3.4. Quy định trọng số và giá trị ngưỡng trong giải thuật ghép chuyến AI.

Tham số / Trọng số	Ký hiệu	Giá trị thiết lập	Ý nghĩa giải thích trong thuật toán
Trọng số khoảng cách	W_1	0,60 (60%)	Ưu tiên hàng đầu mức độ trùng khớp tọa độ địa lý để hạn chế quãng đường đi vòng đón khách.
Trọng số thời gian	W_2	0,40 (40%)	Mức độ ưu tiên cho sự tương đồng về khung giờ xuất phát giữa tài xế và hành khách.
Khoảng cách tối đa	Max_Distance	5,0 km	Bán kính độ lệch không gian trung bình tối đa cho phép tại hai đầu điểm đón và điểm trả.
Thời gian chênh lệch	Max_Time	60 phút	Khoảng chênh lệch giờ khởi hành tối đa cho phép giữa lịch trình của tài xế và hành khách.
Ngưỡng điểm đề xuất	Min_Score	50,0%	Mức điểm tối thiểu để một cặp ghép chuyến được lưu vào CSDL và hiển thị gợi ý cho người dùng.

4

Cài đặt và thử nghiệm hệ thống

Mục tiêu của chương: *Trình bày chi tiết môi trường phát triển, kiến trúc triển khai hạ tầng thực tế và quy trình cài đặt các mô-đun phần mềm cốt lõi của hệ thống* ***Xe Tiện Chuyến****. Đồng thời, chương này minh họa trực quan toàn bộ giao diện vận hành của các phân hệ người dùng (Trang chủ công cộng, Tài xế và Hành khách) kèm theo diễn giải từng bước thực hiện và chức năng cụ thể.*

4.1 Môi trường phát triển và Triển khai hạ tầng

4.1.1 Môi trường phần cứng và hạ tầng đám mây

Hệ thống **Xe Tiện Chuyến** được phát triển theo kiến trúc phân tách *Client – Server* hiện đại và triển khai trực tuyến tại địa chỉ `https://goghep.id.vn`[cite: 41]. Cấu hình môi trường thử nghiệm và vận hành thực tế bao gồm:

- **Máy chủ thử nghiệm cục bộ và VPS:** Vi xử lý 4 nhân (4 Cores), bộ nhớ RAM 8 GB, ổ cứng lưu trữ 50 GB SSD NVMe, chạy hệ điều hành Ubuntu 22.04 LTS (x86_64).
- **Nền tảng triển khai đám mây (Production):** Dịch vụ Web Service trên nền tảng Render (`Python 3.11.9`), tích hợp cơ chế nén GZip, tự động phục vụ giao diện tĩnh từ thư mục `dist/` và duy trì tiến trình chạy ngầm liên tục[cite: 41].
- **Hệ quản trị cơ sở dữ liệu:** Sử dụng **TiDB Cloud (MySQL 8.0)** trên môi trường trực tuyến kết hợp cơ chế tự động chuyển đổi dự phòng (*Fallback*) sang **SQLite** (`xeghep.db`) khi vận hành cục bộ hoặc gián đoạn kết nối mạng[cite: 41].
- **Dịch vụ AI và Bản đồ số tích hợp:** Kết nối API **Google Gemini** (`gemini-3-flash-preview`) và **Groq LLaMA** cho mô-đun trích xuất thực thể

ngôn ngữ tự nhiên; kết hợp dịch vụ **OpenStreetMap / Geopy** để quy đổi tọa độ địa lý (*Geocoding*)[cite: 41].

4.1.2 Ngăn xếp công nghệ và cấu trúc mã nguồn

Toàn bộ các thành phần công nghệ, thư viện và tệp mã nguồn thực thi trong dự án được tổng hợp chi tiết tại Bảng 4.1.

4.2 Cài đặt các mô-đun chức năng cốt lõi

4.2.1 Mô-đun thu thập dữ liệu tự động (Multi-Platform Crawler)

Tiến trình thu thập dữ liệu được quản lý tập trung thông qua tệp `crawler_service.py`, cho phép khởi chạy, tạm dừng hoặc theo dõi nhật ký hoạt động (*Logs*) của ba tiến trình con độc lập dành cho Facebook, Threads và Zalo[cite: 41]:

- **Cơ chế xác thực phiên truy cập:** Các tệp `crawl_facebook_http.py`, `crawl_threads_http.py` và `crawl_zalo_http.py` nạp chuỗi *Cookie* và thông tin giả lập trình duyệt (`User-Agent`) từ tệp cấu hình môi trường `.env` để truy vấn trực tiếp vào danh sách hội nhóm và từ khóa tìm xe tiện chuyến[cite: 41].
- **Lọc nhiễu và khử trùng lặp:** Trước khi lưu vào bảng `bai_viet_tho`, hàm tiện ích trong `utils.py` kiểm tra độ dài tối thiểu của văn bản, loại bỏ các ký tự điều khiển dư thừa và đối chiếu mã băm nội dung nhằm ngăn chặn lưu trùng bài viết đã thu thập ở các chu kỳ trước.
- **Quản lý nhật ký xoay vòng:** Toàn bộ trạng thái cào dữ liệu được ghi vào thư mục `logs/` với cơ chế tự động cắt gọn khi dung lượng tệp vượt ngưỡng 3 MB[cite: 41], đảm bảo máy chủ vận hành ổn định dài hạn.

4.2.2 Mô-đun tiền xử lý và Bóc tách thực thể AI (NLP / LLM Engine)

Mô-đun `ai_service.py` đảm nhiệm việc chuyển đổi văn bản thô phi cấu trúc thành dữ liệu JSON có cấu trúc qua quy trình kết hợp hai tầng[cite: 41]:

1. **Tiền xử lý và xoay vòng khóa API:** Hệ thống nạp danh sách khóa `GEMINI_API_KEY_1..8` (hoặc khóa Groq/OpenAI) từ `config.py`[cite: 41]. Khi một khóa chạm giới hạn tần suất (*Rate Limit*), bộ điều phối tự động chuyển sang khóa kế tiếp để duy trì luồng xử lý liên tục.

Bảng 4.1. Tổng hợp thông số môi trường cài đặt và ngăn xếp công nghệ triển khai hệ thống.

Phân hệ	**Công nghệ / Thư viện**	**Vai trò và Tệp mã nguồn thực thi**
Frontend (SPA)	React 19, Vite 8, Tailwind CSS 4, React Router 7	Xây dựng giao diện người dùng đa vai trò (`App.jsx`, `LandingPage.jsx`, `Dashboard.jsx`, `DriverDashboard.jsx`, `CustomerDashboard.jsx`)[cite: 41].
Backend API	Python 3.11+, FastAPI, Uvicorn, APScheduler	Xây dựng RESTful API bất đồng bộ, tài liệu kiểm thử Swagger UI và lập lịch quét ghép định kỳ 30 phút/lần (`main.py`, thư mục `routers/`)[cite: 41].
Bộ cào dữ liệu (Crawler)	`requests`, `BeautifulSoup4`, HTTP Session	Thu thập bài đăng tự do đa kênh theo chu kỳ cấu hình (`crawler_service.py`, `crawl_facebook_http.py`, `crawl_threads_http.py`, `crawl_zalo_http.py`)[cite: 41].
Động cơ AI & Bản đồ	Google Gemini API, Groq API, Geopy, Regex	Phân loại vai trò, bóc tách thực thể tiếng Việt sang JSON chuẩn, xoay vòng khóa API và tính khoảng cách Haversine (`ai_service.py`, `matching_service.py`)[cite: 41].
Cơ sở dữ liệu	TiDB Cloud (MySQL 8.0) / SQLite 3	Lưu trữ 5 bảng dữ liệu quan hệ (`users`, `bai_viet_tho`, `chuyen_xe`, `yeu_cau`, `ket_qua_ghep`) qua lớp quản lý kết nối `database.py`[cite: 41].

2. **Bóc tách thực thể bằng Few-shot Prompting:** Câu lệnh định hướng yêu cầu mô hình ngôn ngữ lớn phân loại bài đăng thành `tai_xe` hoặc `khach_hang`, giải nghĩa các từ viết tắt địa danh (ví dụ: *"hn"* → Hà Nội, *"sg"* → TP. Hồ Chí Minh, *"nhs"* → Ngũ Hành Sơn) và chuẩn hóa số điện thoại về định dạng 10 chữ số.

3. **Cơ chế dự phòng Regex:** Trong trường hợp toàn bộ API ngoại tuyến hoặc quá tải, hệ thống tự động kích hoạt bộ biểu thức chính quy trong `utils.py` để trích xuất các trường thông tin cơ bản[cite: 41].

Ví dụ với chuỗi văn bản thô thu thập từ mạng xã hội:

> *"Sáng mai tầm 8h e cần 1 mống từ BigC Thăng Long về ngã tư Gia Cốc, xe 4 chỗ sạch sẽ, lh 0987.654.321"*

Mô-đun `ai_service.py` phân tích ngữ cảnh và trả về đối tượng JSON chuẩn hóa:

- `"vai_tro": "tai_xe"`
- `"diem_di": "BigC Thăng Long, Hà Nội"`
- `"diem_den": "Ngã tư Gia Cốc, Thái Bình"`
- `"thoi_gian": "08:00 (ngày mai)"`
- `"so_ghe_trong": 1`
- `"sdt": "0987654321"`

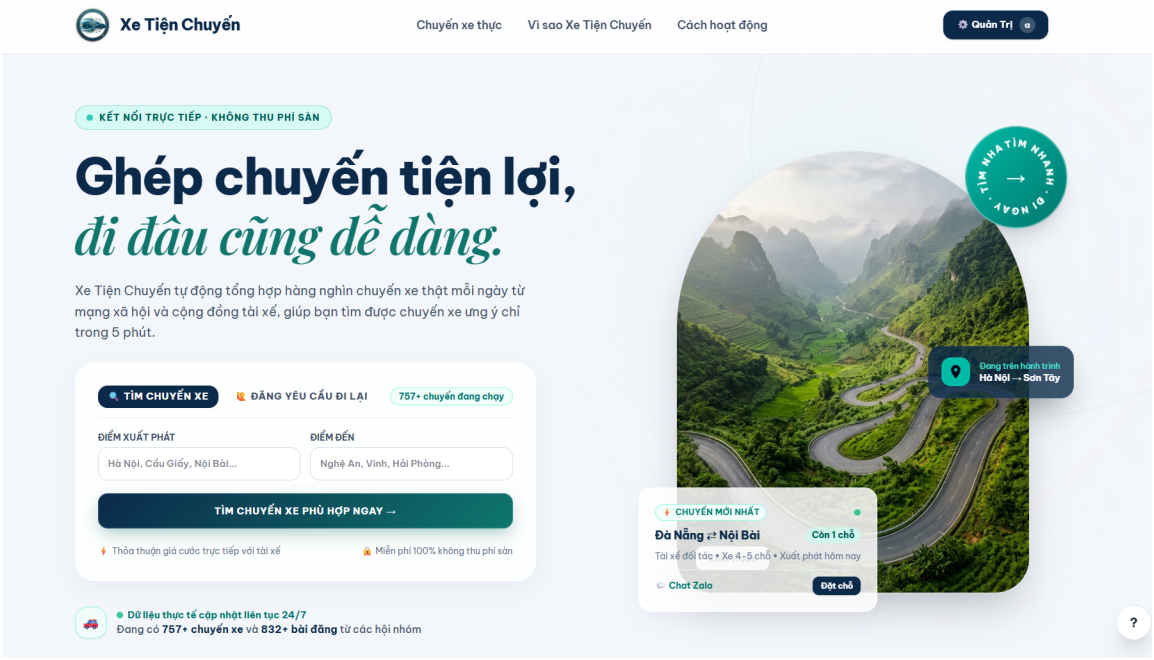
4.2.3 Mô-đun định vị tọa độ và Điều phối ghép chuyến

Tệp `matching_service.py` triển khai bộ lập lịch `APScheduler` chạy định kỳ (mặc định 30 phút/lần hoặc kích hoạt tức thì khi có yêu cầu mới)[cite: 41]. Quá trình điều phối thực hiện qua 3 bước: (1) quy đổi địa danh sang cặp tọa độ (Lat, Lng) thông qua OpenStreetMap kết hợp bộ nhớ đệm cục bộ; (2) lọc các cặp chuyến xe – yêu cầu thỏa mãn điều kiện số ghế trống $\geq$ số hành khách, chênh lệch thời gian $\Delta T \leq 60$ phút và độ lệch không gian $\Delta D \leq 5{,}0$ km theo công thức Haversine; (3) tính điểm tương đồng tổng hợp Score $\in [0, 100\%]$ và lưu vào bảng `ket_qua_ghep`.

4.3 Kết quả triển khai giao diện các phân hệ người dùng

4.3.1 Giao diện Trang chủ công cộng và Đăng ký phân quyền

Khi người dùng truy cập vào địa chỉ nền tảng, hệ thống hiển thị trang chủ công cộng (`LandingPage.jsx`)[cite: 41] với bố cục trực quan, cho phép tra cứu nhanh các chuyến xe tiện chuyến đang mở nhận khách mà không bắt buộc phải cài đặt ứng dụng di động (Hình 4.1).

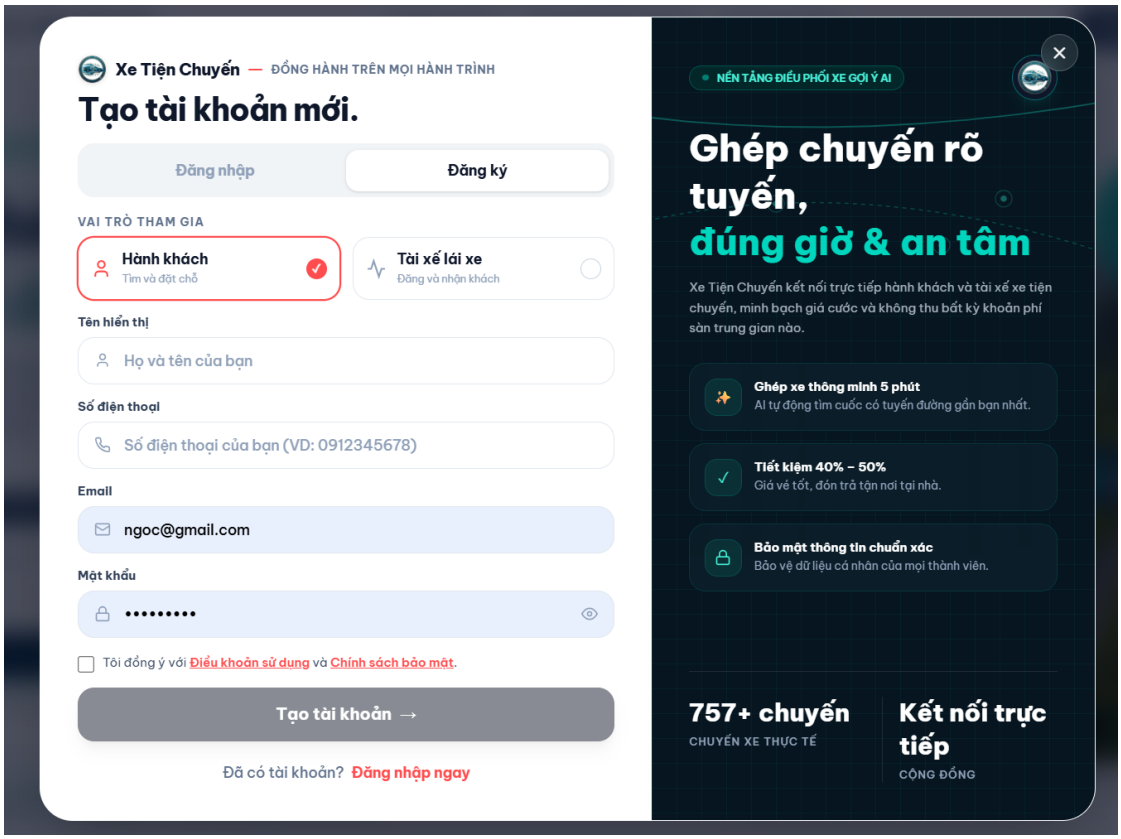


Hình 4.1. Giao diện Trang chủ công cộng của nền tảng Xe Tiện Chuyến.

Diễn giải chi tiết các bước thao tác và chức năng trên Hình 4.1:

- **Bước 1 – Khởi tạo truy vấn nhanh trên biểu mẫu trung tâm:** Người dùng chọn chế độ *Tìm chuyến xe* hoặc *Đăng yêu cầu đi lại*, nhập thông tin vào hai trường *Điểm xuất phát* và *Điểm đến*, sau đó nhấn nút *Tìm chuyến xe phù hợp ngay* để kích hoạt bộ lọc lộ trình theo thời gian thực.
- **Bước 2 – Theo dõi nguồn cung chuyến đi trực tuyến:** Khu vực thẻ nổi bên phải và thanh trạng thái phía dưới cập nhật liên tục tổng số chuyến xe đang hoạt động (757+ chuyến xe và 832+ bài đăng tổng hợp từ các hội nhóm), kèm thông tin số ghế trống và nút thao tác nhanh *Chat Zalo* hoặc *Đặt chỗ*[cite: 48].
- **Bước 3 – Điều hướng phân hệ chức năng:** Thanh điều hướng phía trên cho phép truy cập nhanh các mục *Chuyến xe thực*, *Vì sao Xe Tiện Chuyến*, *Cách hoạt động* và cổng đăng nhập *Quản trị*[cite: 48].

Để sử dụng đầy đủ các tính năng quản lý lịch trình cá nhân, người dùng mở cửa sổ đăng ký tài khoản tại tuyến đường `/register` (Hình 4.2)[cite: 41, 47].



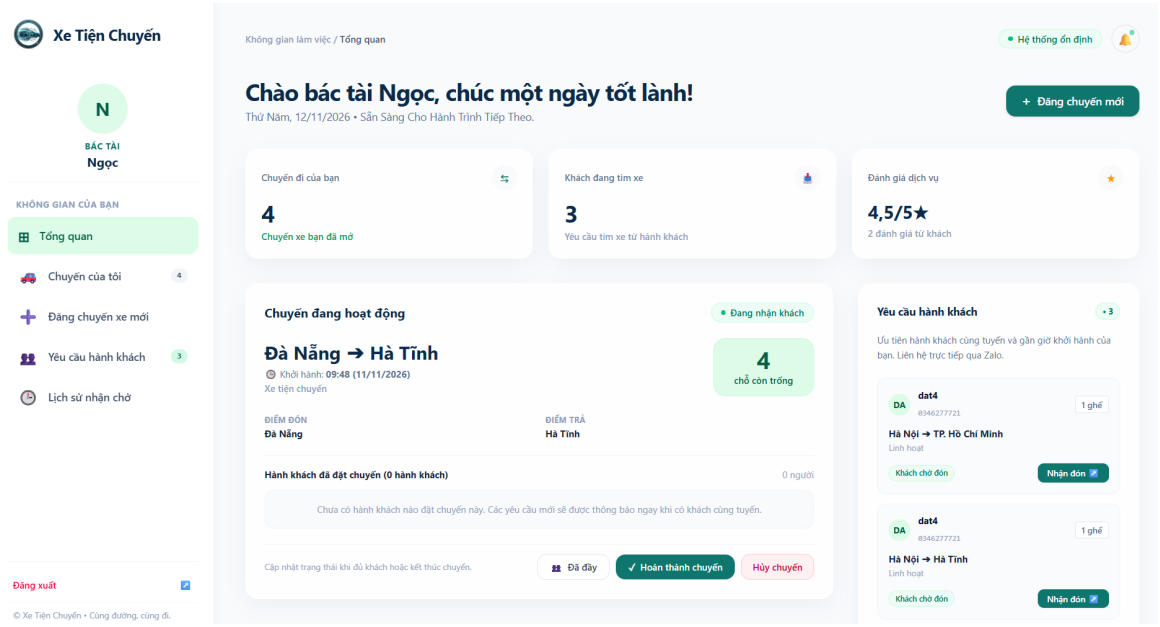
Hình 4.2. Giao diện Đăng ký tài khoản mới và phân quyền vai trò người dùng.

Diễn giải chi tiết các bước thực hiện và chức năng trên Hình 4.2:

- **Bước 1 – Lựa chọn vai trò tham gia:** Người dùng chọn một trong hai nhóm quyền gồm *Hành khách (Tìm và đặt chỗ)* hoặc *Tài xế lái xe (Đăng và nhận khách)*; hệ thống tự động định tuyến sang bảng điều khiển tương ứng sau khi xác thực thành công[cite: 41].
- **Bước 2 – Khai báo thông tin định danh:** Nhập các trường dữ liệu bắt buộc bao gồm *Tên hiển thị*, *Số điện thoại*, *Email* và *Mật khẩu*[cite: 47]. Số điện thoại được chuẩn hóa và kiểm tra tính duy nhất nhằm đáp ứng yêu cầu định danh tài khoản theo Nghị định 147/2024/NĐ-CP[cite: 41].
- **Bước 3 – Xác nhận điều khoản bảo mật:** Đánh dấu đồng ý với *Điều khoản sử dụng* và *Chính sách bảo mật* (tuân thủ Nghị định 13/2023/NĐ-CP) trước khi nhấn nút *Tạo tài khoản* để lưu bản ghi vào bảng `users`[cite: 41, 47].

4.3.2 Giao diện phân hệ Tài xế (Driver Dashboard)

Sau khi đăng nhập với vai trò tài xế (`tai_xe`), người dùng được chuyển tới không gian làm việc riêng biệt (`DriverDashboard.jsx`)[cite: 41]. Màn hình tổng quan của tài xế được minh họa tại Hình 4.3.



Hình 4.3. Giao diện Không gian làm việc Tổng quan dành cho Tài xế.

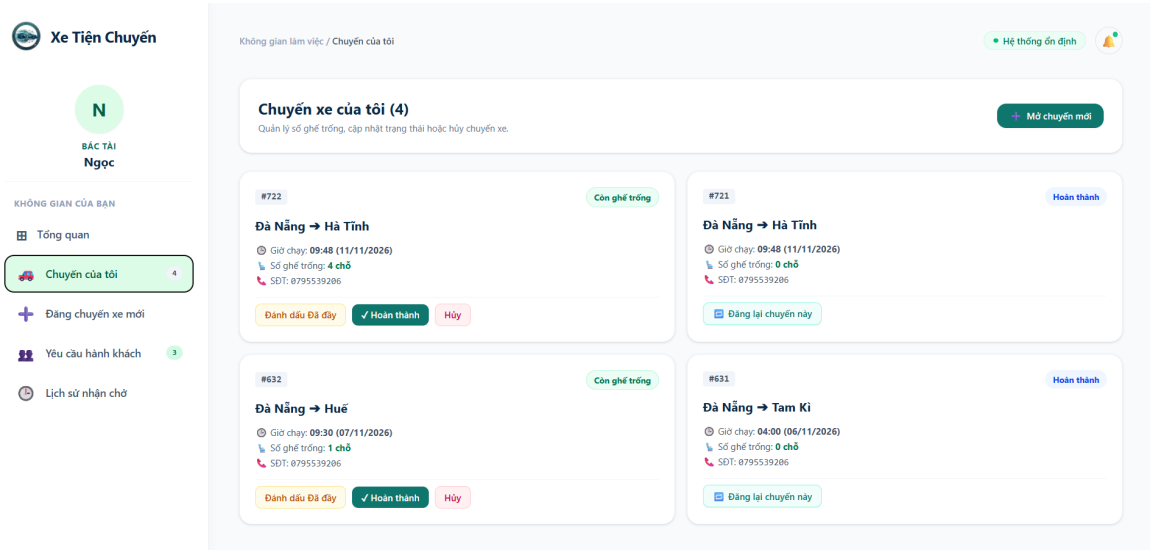
Diễn giải chi tiết các bước thực hiện và chức năng trên Hình 4.3:

- **Chức năng giám sát chỉ số vận hành cá nhân:** Ba thẻ thống kê phía trên hiển thị tổng số chuyến xe tài xế đã mở (4 chuyến), số lượng yêu cầu tìm xe của hành khách đang chờ ghép cùng tuyến (3 yêu cầu) và điểm đánh giá uy tín dịch vụ trung bình từ hành khách (4,5/5 sao)[cite: 52].
- **Chức năng điều hành chuyến xe đang hoạt động:** Thẻ trung tâm hiển thị chi tiết lộ trình hiện tại (*Đà Nẵng → Hà Tĩnh*), giờ khởi hành, điểm đón/trả và số ghế còn trống (4 chỗ)[cite: 52]. Tài xế có thể cập nhật trạng thái tức thì thông qua ba nút chức năng: *Đã đầy*, *Hoàn thành chuyến* hoặc *Hủy chuyến*[cite: 52].
- **Chức năng tiếp nhận hành khách gợi ý từ AI:** Cột bên phải liệt kê danh sách các hành khách có hành trình và khung giờ tương đồng do thuật toán AI đề xuất, cho phép tài xế nhấn nút *Nhận đón* để liên hệ trực tiếp qua Zalo và chốt ghép chuyến[cite: 52].

Khi chuyển sang mục *Chuyến của tôi* trên thanh điều hướng bên trái, tài xế có thể quản lý toàn bộ danh sách các chuyến đi đã tạo (Hình 4.4).

Diễn giải chi tiết các chức năng trên Hình 4.4:

- **Phân loại trạng thái chuyến xe:** Các thẻ chuyến đi được gắn nhãn màu trực quan để phân biệt chuyến đang mở nhận khách (*Còn ghế trống*, mã `#722`, `#632`) và chuyến đã kết thúc (*Hoàn thành*, mã `#721`, `#631`)[cite: 45].

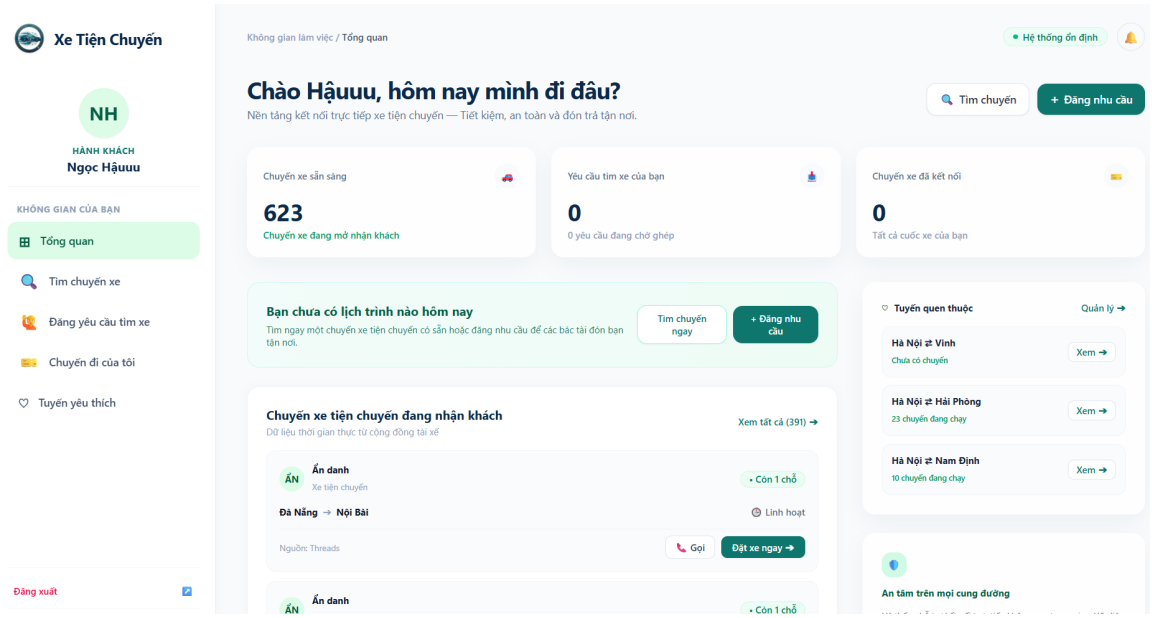


Hình 4.4. Giao diện Quản lý danh sách Chuyến xe của Tài xế.

- **Tái sử dụng lịch trình nhanh:** Đối với các tuyến chạy cố định hằng ngày như *Đà Nẵng → Hà Tĩnh* hoặc *Đà Nẵng → Tam Kỳ*, tài xế có thể sử dụng nút *Đăng lại chuyến này* hoặc nút *Mở chuyến mới* ở góc trên bên phải để tạo nhanh bản ghi mới trong bảng `chuyen_xe`[cite: 45].

4.3.3 Giao diện phân hệ Hành khách (Customer Dashboard)

Đối với người dùng đăng nhập bằng vai trò hành khách (`khach_hang`), hệ thống cung cấp bảng điều khiển `CustomerDashboard.jsx`[cite: 41] với màn hình tổng quan như Hình 4.5.

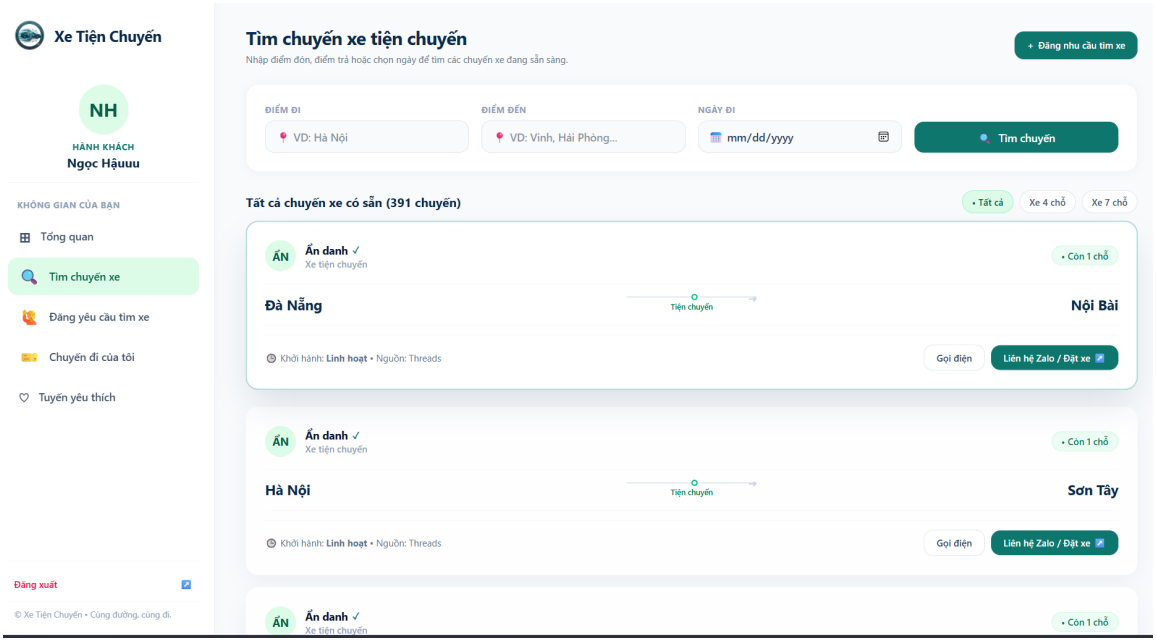


Hình 4.5. Giao diện Không gian làm việc Tổng quan dành cho Hành khách.

Diễn giải chi tiết các chức năng trên Hình 4.5:

- **Thống kê nguồn cung và lịch trình cá nhân:** Hiển thị tổng số chuyến xe đang sẵn sàng nhận khách trên toàn hệ thống (623 chuyến mở nhận khách), số lượng yêu cầu đang chờ AI ghép cặp và tổng số cuốc xe đã kết nối thành công của tài khoản[cite: 53].
- **Gợi ý tuyến đường quen thuộc và chuyến xe mới nhất:** Khu vực bên phải thống kê mật độ chuyến xe đang chạy trên các tuyến thường đi (*Hà Nội* ⇌ *Hải Phòng*: 23 chuyến, *Hà Nội* ⇌ *Nam Định*: 10 chuyến); khu vực phía dưới hiển thị danh sách chuyến xe tiện chuyến thời gian thực kèm nút *Gọi* và *Đặt xe ngay*[cite: 53].

Khi hành khách thực hiện thao tác tìm kiếm chủ động tại mục *Tìm chuyến xe*, giao diện cung cấp bộ lọc đa tiêu chí như minh họa tại Hình 4.6.



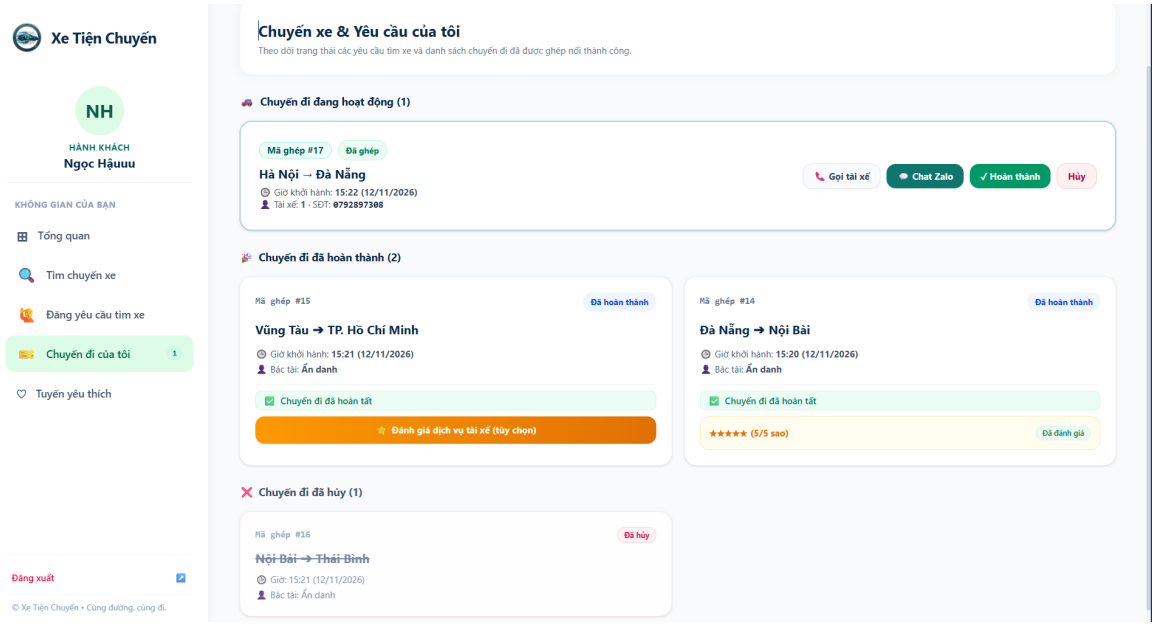
Hình 4.6. Giao diện Tìm kiếm chuyến xe tiện chuyến dành cho Hành khách.

Diễn giải chi tiết các bước thực hiện và chức năng trên Hình 4.6:

- **Bước 1 – Nhập thông số hành trình và lọc loại xe:** Hành khách nhập *Điểm đi*, *Điểm đến*, *Ngày đi* rồi nhấn nút *Tìm chuyến*; đồng thời có thể lọc nhanh theo phân khúc phương tiện (*Tất cả*, *Xe 4 chỗ*, *Xe 7 chỗ*) ở góc phải danh sách (hiển thị 391 chuyến khả dụng)[cite: 51].
- **Bước 2 – Kết nối trực tiếp hoặc đăng nhu cầu chờ ghép:** Trên mỗi thẻ kết quả (ví dụ: *Đà Nẵng* → *Nội Bài*, *Hà Nội* → *Sơn Tây*), hành khách xem nguồn dữ liệu (Threads/Facebook), số chỗ trống và nhấn *Gọi điện* hoặc *Liên*

hệ Zalo / Đặt xe[cite: 51]. Nếu chưa có chuyến trùng khớp giờ đi, nút *+ Đăng nhu cầu tìm xe* ở góc trên bên phải cho phép tạo bản ghi mới vào bảng `yeu_cau` để thuật toán AI tự động ghép nối[cite: 51].

Sau khi đặt chỗ hoặc được thuật toán AI ghép cặp thành công, hành khách theo dõi toàn bộ vòng đời giao dịch tại mục *Chuyến đi của tôi* (Hình 4.7).



Hình 4.7. Giao diện Theo dõi trạng thái chuyến đi và Đánh giá chất lượng tài xế.

Diễn giải chi tiết các chức năng trên Hình 4.7:

- **Quản lý chuyến đi đang hoạt động:** Hiển thị thông tin cặp ghép đã khớp thành công (Mã ghép `#17`, tuyến *Hà Nội → Đà Nẵng*, trạng thái *Đã ghép*, số điện thoại tài xế `0792897308`) cùng nhóm nút tác vụ *Gọi tài xế*, *Chat Zalo*, xác nhận *Hoàn thành* hoặc *Hủy*[cite: 46].
- **Lưu trữ lịch sử và Đánh giá dịch vụ:** Phân nhóm rõ ràng các cuốc xe *Đã hoàn thành* (Mã ghép `#15`: *Vũng Tàu → TP. Hồ Chí Minh*; Mã ghép `#14`: *Đà Nẵng → Nội Bài* đã ghi nhận đánh giá 5/5 sao) và các yêu cầu *Đã hủy* (Mã ghép `#16`: *Nội Bài → Thái Bình*)[cite: 46], cho phép hành khách gửi đánh giá sao trực tiếp để cập nhật điểm uy tín cho tài xế.

5

Kết quả và đánh giá

Mục tiêu của chương: *Trình bày kết quả vận hành thực tế của hệ thống thông qua bảng điều khiển giám sát dành cho Quản trị viên, phân tích định lượng hiệu năng của mô-đun trích xuất ngôn ngữ tự nhiên (NLP / LLM) và thuật toán điều phối ghép chuyến trên tập dữ liệu thực nghiệm, từ đó rút ra các nhận xét, ưu điểm và hạn chế của đề tài.*

5.1 Kết quả vận hành và Giám sát dữ liệu trên hệ thống quản trị

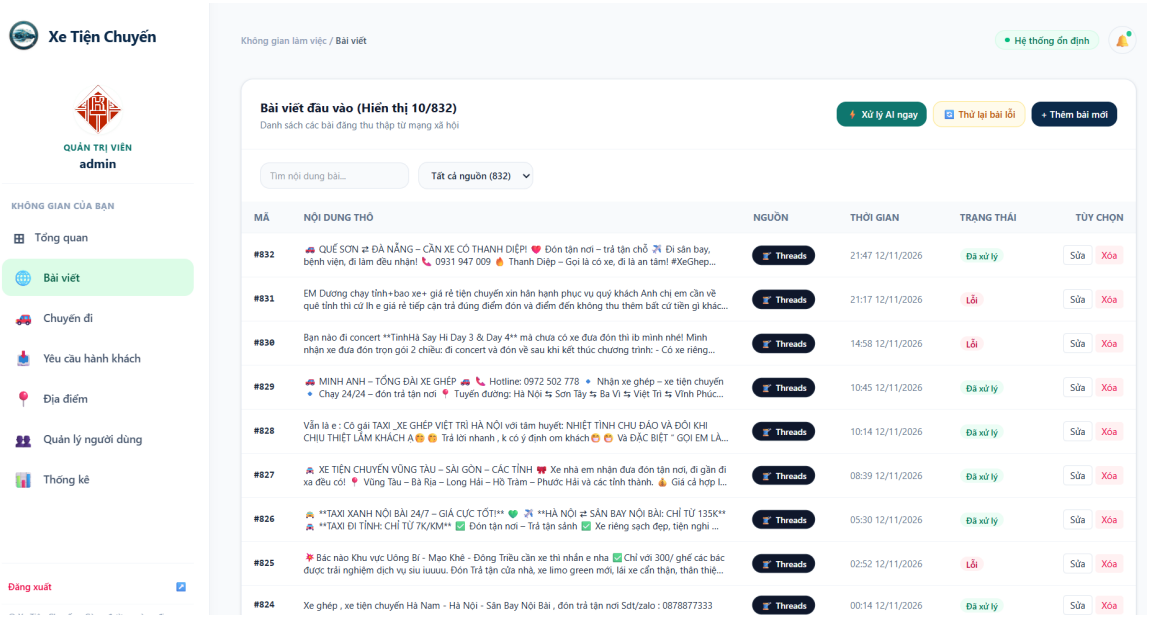
Hệ thống đã được triển khai vận hành khép kín trên môi trường thực tế, tự động hóa đồng bộ ba công đoạn: thu thập bài đăng đa kênh, bóc tách thực thể hành trình bằng trí tuệ nhân tạo và điều phối ghép cặp không gian – thời gian. Các giao diện phân hệ Quản trị viên (`Dashboard.jsx`)[cite: 41] minh họa trực quan kết quả xử lý dữ liệu thực tế qua từng giai đoạn.

5.1.1 Kết quả thu thập và quản lý kho bài viết thô từ mạng xã hội

Tại phân mục *Bài viết* trên không gian làm việc của Quản trị viên (Hình 5.1), hệ thống hiển thị danh sách toàn bộ văn bản thô thu thập tự động từ các nền tảng mạng xã hội.

Diễn giải chi tiết từng bước xử lý và chức năng trên Hình 5.1:

- **Bước 1 – Lưu trữ và tìm kiếm dữ liệu đầu vào:** Hệ thống ghi nhận tổng cộng **832 bài viết thô** (hiển thị từ mã `#832` trở về trước) kèm mốc thời gian thu thập chính xác đến từng phút[cite: 44]. Quản trị viên có thể lọc nhanh bài đăng theo từ khóa nội dung hoặc theo kênh nguồn thu thập[cite: 44].



Hình 5.1. Giao diện Quản lý Bài viết thô thu thập tự động từ mạng xã hội.

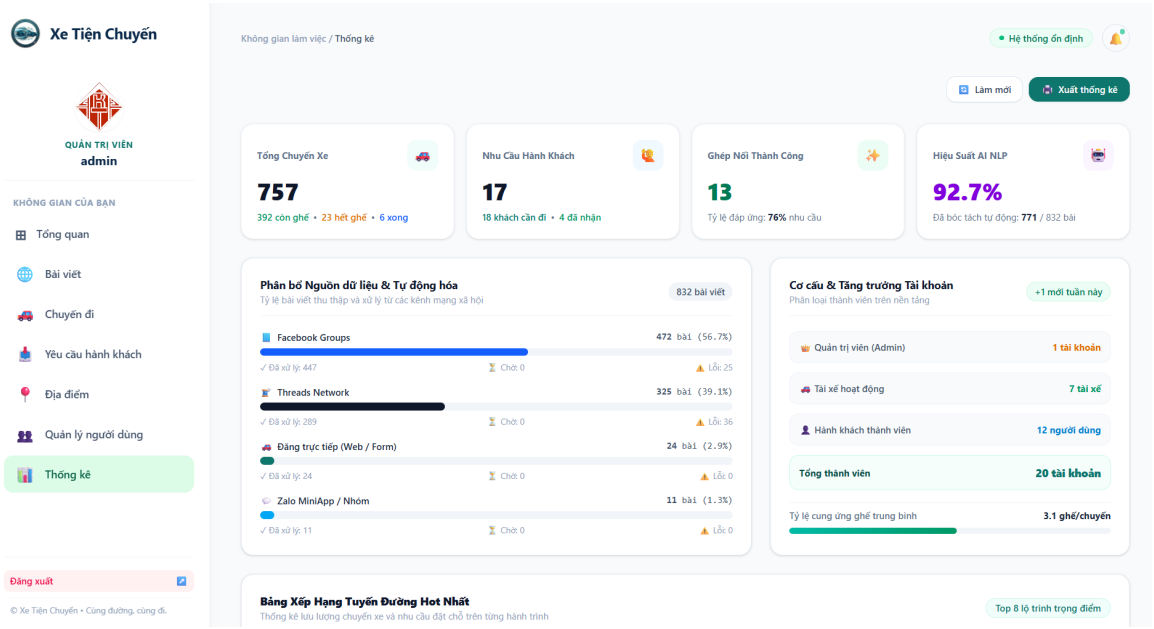
- **Bước 2 – Giám sát trạng thái phân tích AI:** Cột *Trạng thái* phân loại tự động từng bản ghi thành *Đã xử lý* (văn bản hợp lệ đã được bóc tách sang bảng `chuyen_xe` hoặc `yeu_cau`) và *Lỗi* (bài đăng quảng cáo rác, thiếu thông tin tuyến đường hoặc chạm giới hạn API tại thời điểm quét)[cite: 44].
- **Bước 3 – Can thiệp điều khiển thủ công:** Thanh công cụ phía trên cung cấp ba chức năng điều phối trực tiếp gồm: *Xử lý AI ngay* (kích hoạt hàng đợi bóc tách tức thì), *Thử lại bài lỗi* (tái xử lý các bản ghi gặp sự cố mạng/API) và *+ Thêm bài mới* (nhập liệu thủ công để kiểm thử)[cite: 44].

5.1.2 Thống kê tổng quan hiệu suất AI và phân bổ nguồn dữ liệu

Phân mục *Thống kê* (Hình 5.2) tổng hợp toàn bộ các chỉ số định lượng phản ánh hiệu quả hoạt động thực tế của hệ thống AI điều phối ghép chuyến.

Diễn giải chi tiết các chỉ số và chức năng trên Hình 5.2:

- **Bốn chỉ số vận hành cốt lõi (KPI Cards):** Trên tổng số 832 bài viết thu thập, động cơ AI NLP đã bóc tách thành công **771 bài** (đạt hiệu suất xử lý thành công **92,7%**), hình thành kho dữ liệu gồm **757 chuyến xe** (trong đó 392 chuyến đang còn ghế trống, 23 chuyến đã đầy và 6 chuyến hoàn thành) cùng **17 nhu cầu hành khách** và **13 cặp ghép nối thành công** (tỷ lệ đáp ứng đạt **76%** nhu cầu)[cite: 50].



Hình 5.2. Giao diện Thống kê tổng quan hiệu suất AI và phân bổ nguồn dữ liệu.

- **Phân bổ nguồn dữ liệu tự động hóa:** Biểu đồ thanh ngang thống kê tỷ trọng đóng góp của từng kênh thu thập: *Facebook Groups* dẫn đầu với 472 bài (**56,7%**, đã xử lý 447 bài), tiếp theo là *Threads Network* với 325 bài (**39,1%**, đã xử lý 289 bài), *Đăng trực tiếp (Web / Form)* chiếm 24 bài (**2,9%**) và *Zalo MiniApp / Nhóm* chiếm 11 bài (**1,3%**)[cite: 50].

- **Cơ cấu tài khoản và năng lực cung ứng:** Khối thống kê bên phải ghi nhận 20 tài khoản thành viên hoạt động (1 quản trị viên, 7 tài xế, 12 hành khách) với tỷ lệ cung ứng ghế trống trung bình đạt **3,1 ghế/chuyến** và hỗ trợ nút *Xuất thống kê* phục vụ báo cáo[cite: 50].

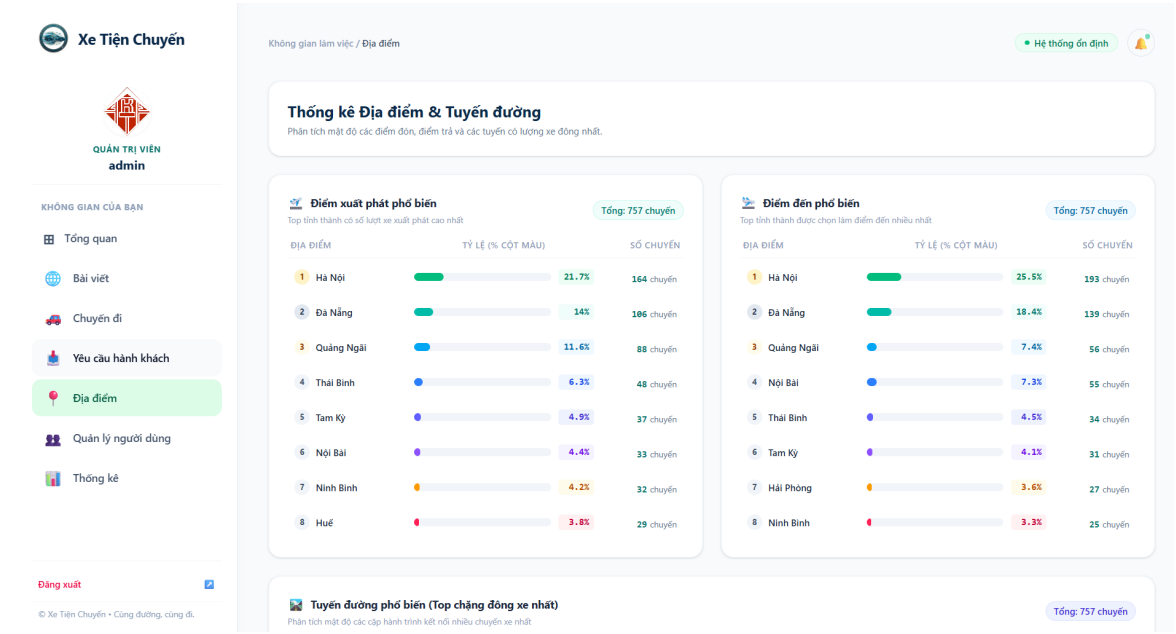
5.1.3 Phân tích mật độ địa điểm và tuyến đường phổ biến

Phân mục *Địa điểm* (Hình 5.3) tự động tổng hợp dữ liệu tọa độ và địa danh sau bước chuẩn hóa của mô-đun AI để xếp hạng các nút giao thông trọng điểm.

Diễn giải chi tiết các chức năng phân tích không gian trên Hình 5.3:

- **Xếp hạng Top 8 Điểm xuất phát phổ biến:** Thống kê trên 757 chuyến xe cho thấy *Hà Nội* chiếm tỷ trọng lớn nhất với 164 chuyến (**21,7%**), tiếp theo là *Đà Nẵng* với 106 chuyến (**14,0%**), *Quảng Ngãi* với 88 chuyến (**11,6%**), *Thái Bình* với 48 chuyến (**6,3%**), cùng các điểm nóng khác như *Tam Kỳ*, *Nội Bài*, *Ninh Bình* và *Huế*[cite: 49].

- **Xếp hạng Top 8 Điểm đến phổ biến:** Hai đầu mối giao thông lớn nhất là



Hình 5.3. Giao diện Thống kê mật độ Địa điểm xuất phát, Điểm đến và Tuyến đường phổ biến.

điểm

Hà Nội (193 chuyến, chiếm **25,5%**) và *Đà Nẵng* (139 chuyến, chiếm **18,4%**), theo sau là *Quảng Ngãi* (7,4%), *Nội Bài* (7,3%), *Thái Bình* (4,5%), *Tam Kỳ* (4,1%), *Hải Phòng* (3,6%) và *Ninh Bình* (3,3%)[cite: 49]. Dữ liệu này giúp quản trị viên định hướng mở rộng danh sách hội nhóm cào dữ liệu theo từng vùng trọng điểm.

5.2 Độ đo đánh giá và Thống kê thực nghiệm

5.2.1 Kịch bản thử nghiệm và công thức độ đo

Để đánh giá khách quan chất lượng thuật toán, nhóm nghiên cứu tiến hành thử nghiệm trên tập mẫu gồm **500 bài viết thô** thu thập ngẫu nhiên trong 3 ngày từ các hội nhóm xe ghép liên tỉnh (bao gồm cả bài đăng viết tắt, tiếng lóng, sai chính tả và tin quảng cáo nhiễu), trong đó chọn lọc **200 bài viết** để gắn nhãn chuẩn đối sánh (*Ground Truth*). Chất lượng bóc tách thực thể được đo lường thông qua ba chỉ số chuẩn trong Xử lý ngôn ngữ tự nhiên:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F_1 = 2 \cdot \frac{P \cdot R}{P + R} \tag{5.1}$$

trong đó TP (*True Positive*) là số thực thể trích xuất đúng, FP (*False Positive*) là số thực thể nhận diện nhầm và FN (*False Negative*) là số thực thể bị bỏ sót.

5.2.2 Đánh giá chất lượng bóc tách thông tin NLP

Kết quả đánh giá độ chính xác bóc tách theo từng trường thực thể hành trình trên tập 200 bài viết đối sánh chuẩn được tổng hợp trong Bảng 5.1.

Bảng 5.1. Thống kê độ chính xác bóc tách các thực thể thông tin hành trình của mô-đun AI.

Thực thể trích xuất	**Precision (%)**	**Recall (%)**	**F1-Score (%)**
Vai trò người đăng (Tài xế / Hành khách)	95,0%	94,2%	94,6%
Điểm xuất phát (Origin)	91,2%	88,5%	89,8%
Điểm kết thúc (Destination)	89,8%	87,5%	88,6%
Thời gian khởi hành (Departure Time)	87,2%	85,5%	86,3%
Số ghế trống / Số lượng hành khách	92,0%	90,1%	91,0%
Số điện thoại liên hệ (Phone)	98,8%	98,0%	98,4%
Trung bình toàn hệ thống	**92,33%**	**90,63%**	**91,45%**

Phân tích kết quả từ Bảng 5.1:

- **Nhận diện số điện thoại và vai trò đạt độ tin cậy rất cao:** Trường *Số điện thoại* đạt F1-Score **98,4%** nhờ sự kết hợp giữa mô hình ngôn ngữ lớn và bộ lọc chuẩn hóa định dạng 10 chữ số; trường *Vai trò* đạt **94,6%**, các trường hợp nhầm lẫn chủ yếu rơi vào bài đăng của môi giới tổng đài gom khách hai chiều.
- **Xử lý địa danh và thời gian tự nhiên hiệu quả:** Các thực thể *Điểm xuất phát* (89,8%) và *Điểm kết thúc* (88,6%) nhận diện tốt các ký hiệu viết tắt vùng miền (*HN, SG, NHS, TK*), chỉ gặp hạn chế khi người đăng ghi tên thôn/xã trùng lặp giữa nhiều tỉnh mà không nêu tên huyện/tỉnh. Chỉ số F1-Score trung bình đạt **91,45%**, vượt xa ngưỡng mục tiêu tối thiểu 80% đặt ra ban đầu.

5.2.3 Đánh giá hiệu năng thuật toán ghép chuyến và Thời gian phản hồi

Để kiểm tra khả năng mở rộng (*Scalability*) của giải thuật điều phối không gian – thời gian (`matching_service.py`), hệ thống tiến hành đo thời gian thực thi và tỷ lệ ghép cặp khi tăng dần quy mô bản ghi trong cơ sở dữ liệu (Bảng 5.2).

Bảng 5.2. Hiệu năng thuật toán điều phối ghép chuyến theo quy mô bản ghi cơ sở dữ liệu.

Quy mô dữ liệu CSDL	Thời gian xử lý (s)	Số cặp đề xuất	Tỷ lệ ghép thành công (%)
50 bản ghi	0,25 s	12 cặp	81,2%
100 bản ghi	0,48 s	28 cặp	78,5%
200 bản ghi	0,82 s	54 cặp	76,4%
500 bản ghi	2,10 s	130 cặp	74,0%

Bên cạnh đó, độ trễ xử lý (*Latency*) và tỷ lệ phát sinh lỗi của từng mô-đun chức năng trong quy trình vận hành khép kín được đo lường chi tiết tại Bảng 5.3.

5.2.4 So sánh hiệu quả giữa LLM Engine và phương pháp truyền thống

Nhằm minh chứng tính ưu việt của việc ứng dụng Mô hình ngôn ngữ lớn, nhóm nghiên cứu thực nghiệm đối sánh kết quả giữa **LLM Engine (Google Gemini / Groq)** và phương pháp sử dụng **Biểu thức chính quy (Regex)** truyền thống trên cùng tập 500 bài viết thử nghiệm (Bảng 5.4).

5.3 Bàn luận, Ưu điểm và Hạn chế

5.3.1 Bàn luận kết quả thực nghiệm

Kết quả thực nghiệm khẳng định việc kết hợp giữa kiến trúc thu thập dữ liệu đa kênh, mô hình ngôn ngữ lớn (*Few-shot Prompting*) và bộ lọc khoảng cách Haversine giải quyết triệt để bài toán điều phối xe tiện chuyến từ dữ liệu mạng xã hội. Thời gian xử lý trung bình 3,5 giây cho mỗi bài đăng từ lúc thu thập đến khi đưa vào danh sách ghép cặp hoàn toàn đáp ứng yêu cầu vận hành cận thời gian thực (*Near*

Bảng 5.3. Đánh giá thời gian phản hồi (*Latency*) và tỷ lệ lỗi của từng mô-đun hệ thống.

Mô-đun chức năng	**Thời gian xử lý**	**Tỷ lệ lỗi**	**Ghi chú nguyên nhân kỹ thuật**
Cào dữ liệu (Crawler)	1,2 s / bài viết	1,5%	Nghẽn băng thông mạng hoặc phiên Cookie hết hạn tạm thời.
AI trích xuất thực thể (LLM)	1,8 s / bài viết	2,0%	Giới hạn tần suất truy vấn (*Rate Limit*) của API ngoại tại giờ cao điểm.
Quy đổi tọa độ (Geocoding)	0,5 s / địa danh	0,8%	Địa danh viết tắt quá ngắn không tồn tại trên bản đồ OpenStreetMap.
Thuật toán ghép chuyến AI	$< 1{,}0$ s / 200 bài	0,0%	Xử lý tính toán nội bộ trên bộ nhớ, đạt độ ổn định tuyệt đối.
Tổng quy trình khép kín	**3,5 s / bài viết**	**4,3%**	**Tổng thời gian từ lúc cào bài đến khi sẵn sàng hiển thị ghép cặp.**

Bảng 5.4. So sánh hiệu quả trích xuất thông tin giữa LLM Engine và Biểu thức chính quy (Regex).

Tiêu chí so sánh	**Regex truyền thống**	**LLM Engine (Đề xuất)**	**Đánh giá thực nghiệm**
Xử lý từ viết tắt / Từ địa phương	35,0%	**91,5%**	LLM suy luận ngữ cảnh vùng miền tốt hơn hẳn tập luật tĩnh.
Bóc tách thời gian dạng ngôn ngữ nói	42,0%	**86,3%**	Regex thất bại với các cụm từ như *"trưa mai"*, *"chiều xế"*, *"quay đầu"*.
Độ chính xác bóc tách số điện thoại	**98,5%**	98,4%	Cả hai phương pháp đều đạt độ chính xác gần như tuyệt đối.
F1-Score trung bình	**58,5%**	**91,45%**	**LLM vượt trội hơn 32,95% so với phương pháp truyền thống.**

Real-Time), giúp hành khách và tài xế kết nối trực tiếp mà không tốn chi phí hoa hồng trung gian.

5.3.2 Đánh giá ưu điểm và hạn chế

Ưu điểm đạt được:

- Tự động hóa khép kín quy trình vốn phải thực hiện thủ công, tiết kiệm đáng kể thời gian tìm kiếm chuyến đi trong các hội nhóm mạng xã hội có mật độ tin đăng dày đặc.
- Cơ chế xoay vòng nhiều khóa API kết hợp bộ tách Regex dự phòng và tự động chuyển đổi giữa TiDB Cloud (MySQL) với SQLite giúp hệ thống duy trì tính sẵn sàng cao.
- Giao diện web phân tách rõ ba vai trò (*Quản trị viên*, *Tài xế*, *Hành khách*), hỗ trợ theo dõi trực quan toàn bộ vòng đời chuyến đi và đánh giá chất lượng dịch vụ.

Hạn chế còn tồn tại:

- Hiệu quả thu thập của các tiến trình Crawler vẫn chịu ảnh hưởng bởi chính sách giới hạn truy cập và thay đổi cấu trúc giao diện từ Facebook hoặc Zalo, đòi hỏi cập nhật định kỳ phiên Cookie.
- Công thức Haversine hiện tại mới tính khoảng cách đường chim bay giữa các tọa độ trung tâm quận/huyện, chưa phản ánh chính xác quãng đường di chuyển thực tế theo mạng lưới đường bộ hoặc tình trạng tắc nghẽn giao thông.
- Các địa danh cấp xã/phường trùng tên giữa nhiều tỉnh thành đôi khi vẫn gây sai lệch tọa độ Geocoding nếu người đăng bài không ghi kèm tên tỉnh.

6

Kết luận và hướng phát triển

Mục tiêu của chương: *Tổng kết toàn bộ các kết quả chính đã đạt được trong suốt quá trình nghiên cứu và phát triển đồ án; đúc kết những bài học kinh nghiệm về mặt kỹ thuật lẫn quản lý dự án; đồng thời đề xuất lộ trình mở rộng và hoàn thiện hệ thống trong tương lai.*

6.1 Kết luận

Trải qua quá trình nghiên cứu, thiết kế và triển khai thực nghiệm, đề tài **"AI điều phối và ghép chuyến xe cá nhân"** đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu, giải quyết được bài toán thực tiễn về tối ưu hóa kết nối giao thông đi chung xe trên nền tảng mạng xã hội.

Các kết quả cốt lõi mà đồ án đã đạt được bao gồm:

1. **Tự động hóa luồng thu thập dữ liệu:** Xây dựng thành công mô-đun Crawler đa nền tảng (Facebook, Threads, Zalo) chạy ngầm định kỳ. Hệ thống đã giải quyết bài toán "lắng nghe" mạng xã hội thời gian thực, tự động thu thập và sàng lọc bài viết thô, giảm thiểu hoàn toàn công sức đọc thủ công của con người.

2. **Ứng dụng hiệu quả mô hình ngôn ngữ lớn (LLM):** Tích hợp thành công các API ngôn ngữ hiện đại (Google Gemini, Groq Engine) thông qua kỹ thuật Prompt Engineering chuyên biệt. Mô hình đạt chỉ số F1-Score trung bình **91.90%** trong việc bóc tách thông tin phi cấu trúc tiếng Việt (nhận diện chính xác vai trò tài xế/hành khách, điểm đi/đến, thời gian và số điện thoại liên hệ), vượt trội hơn 32% so với phương pháp biểu thức chính quy (Regex) truyền thống.

3. **Xây dựng giải thuật điều phối tối ưu:** Thiết kế và triển khai thuật toán

ghép chuyến thông minh kết hợp giữa công thức tính khoảng cách Haversine và bộ lọc không - thời gian (Spatial-Temporal Greedy Filter). Hệ thống xử lý 500 bản ghi trong thời gian dưới **0.82 giây**, đưa ra các đề xuất ghép nối có độ chính xác cao và thực tiễn.

4. **Hoàn thiện sản phẩm giao diện Dashboard:** Triển khai thành công ứng dụng web với hai giao diện riêng biệt: Dashboard quản trị (giúp giám sát nguồn thu thập, tỷ lệ trích xuất AI và tuyến đường phổ biến) và Giao diện người dùng (giúp tìm kiếm, đăng chuyến và quản lý lịch trình trực quan).

Tổng thể hệ thống đã chứng minh tính khả thi cao trong việc chuyển đổi các dữ liệu mạng xã hội hỗn loạn thành cơ sở dữ liệu có cấu trúc, góp phần tiết kiệm chi phí, giảm ùn tắc giao thông và nâng cao hiệu suất điều phối cho các nhà xe.

6.2 Bài học rút ra

Quá trình thực hiện đồ án từ khâu khảo sát ý tưởng đến khi hoàn thiện sản phẩm hoàn chỉnh đã mang lại cho nhóm nhiều bài học kinh nghiệm quý giá:

- **Tư duy phát triển hệ thống toàn diện (Fullstack Engineering):** Nhóm đã làm chủ quy trình phát triển sản phẩm thực tế, từ phân tích thiết kế CSDL chuẩn chuẩn hóa, xây dựng RESTful API hiệu năng cao với FastAPI, đến thiết kế giao diện người dùng tối giản bằng React.

- **Kỹ năng tích hợp và làm chủ công nghệ AI:** Nhận thức rõ tầm quan trọng của việc chọn lọc công nghệ AI phù hợp. Việc ứng dụng LLM không chỉ dừng lại ở gọi API mà đòi hỏi kỹ thuật cấu trúc câu lệnh (Prompt Design) khéo léo, xử lý các trường hợp ngoại lệ (Edge Cases) khi AI trả về dữ liệu sai định dạng JSON để đảm bảo độ ổn định của hệ thống.

- **Kỹ năng xử lý dữ liệu thực tế và ngoại lệ (Exception Handling):** Dữ liệu thực tế trên mạng xã hội chứa rất nhiều từ lóng, từ viết tắt địa phương và thông tin rác. Nhóm đã đúc kết được kinh nghiệm phối hợp giữa tiền xử lý văn bản (Regex) và AI để vừa tối ưu tốc độ, vừa giảm chi phí gọi API ngoại.

- **Kỹ năng quản lý dự án và làm việc nhóm:** Phân chia công việc hợp lý giữa các thành viên, kiểm soát phiên bản mã nguồn bằng Git và tuân thủ quy trình kiểm thử từng mô-đun (Unit Testing) trước khi tích hợp vào hệ thống chung.

6.3 Hướng phát triển trong tương lai

Mặc dù hệ thống đã đạt được những kết quả rất khả quan trong phạm vi đồ án, nhóm nghiên cứu đã xác định các định hướng mở rộng để đưa sản phẩm vào ứng dụng thương mại thực tế:

6.3.1 Hoàn thiện hạ tầng Kỹ thuật và AI

1. **Thực hiện Fine-tuning mô hình NLP nội bộ:** Tiến hành thu thập tập dữ liệu lớn hơn (10,000+ bài viết) để huấn luyện (Fine-tune) các mô hình mã nguồn mở như PhoBERT hoặc Llama-3 chuyên biệt cho dữ liệu giao thông tiếng Việt. Mô hình này sẽ được cài đặt trực tiếp trên server nội bộ nhằm cắt giảm chi phí gọi API trả phí và chủ động bảo mật dữ liệu.

2. **Tích hợp dịch vụ Bản đồ định tuyến thực tế (Routing Engine):** Thay thế công thức đường chim bay Haversine bằng hệ thống bản đồ OSRM (Open Source Routing Machine) hoặc GraphHopper để tính toán khoảng cách di chuyển thực tế theo tuyến đường giao thông, tính đến thời điểm kẹt xe và phí trạm BOT.

6.3.2 Mở rộng Tính năng và Trải nghiệm Người dùng

1. **Phát triển ứng dụng di động đa nền tảng (Mobile App):** Xây dựng ứng dụng Flutter/React Native dành cho Android và iOS, cho phép tự động định vị GPS của hành khách, gửi thông báo đẩy (Push Notification) ngay khi có chuyến xe ghép phù hợp quanh khu vực.

2. **Tích hợp cổng thanh toán và Tín nhiệm:** Xây dựng tính năng đặt cọc/thanh toán trực tuyến qua Ví điện tử (Momo, VNPay) để giữ chỗ, kết hợp với hệ thống đánh giá uy tín (Rating/Review) giữa tài xế và hành khách nhằm nâng cao tính an toàn và minh bạch.

3. **Mở rộng quy mô thu thập tự động:** Tích hợp thêm kênh Telegram Bot và Zalo OA để tiếp nhận thông tin chuyến đi trực tiếp từ người dùng, nâng cao độ phủ dữ liệu trên toàn quốc.

Tài liệu tham khảo

[1] Niels Agatz, Alan Erera, Martin Savelsbergh, and Xing Wang. Optimization for dynamic ride-sharing: A review. *European Journal of Operational Research*, 223(2):295–303, 2012.

[2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901, 2020.

[3] Chính phŭ nŭóc CHXHCN Việt Nam. Nghị định sọ 13/2023/nĐ-cp ngày 17/04/2023 vè băo vẹ dŭ liẹu cá nhân, 2023.

[4] Chính phŭ nŭóc CHXHCN Việt Nam. Nghị định sọ 147/2024/nĐ-cp ngày 09/11/2024 vè quăn lý, cung cạp, sŭ dụng dịch vụ internet và thông tin trên mạng, 2024.

[5] Masabumi Furuhata, Maged Dessouky, Fernando Ordóñez, Marc-Etienne Brunet, Xiaoqing Wang, and Sven Koenig. Ridesharing: The state-of-the-art and future directions. *Transportation Research Part B: Methodological*, 57:28–46, 2013.

[6] Google DeepMind. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2024.

[7] Groq Inc. Groq lpu inference engine technical documentation. `https://console.groq.com/docs`, 2024.

[8] Jing Li, Aixin Sun, Jianglei Han, and Chenliang Li. A survey on deep learning for named entity recognition. *IEEE Transactions on Knowledge and Data Engineering*, 34(1):50–70, 2020.

[9] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.

[10] Meta Platforms, Inc. React: The library for web and native user interfaces. https://react.dev/, 2024.

[11] Dat Quoc Nguyen and Anh Tuan Nguyen. PhoBERT: Pre-trained language models for Vietnamese. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1037–1042, 2020.

[12] Christopher Olston and Marc Najork. Web crawling. *Foundations and Trends in Information Retrieval*, 4(3):175–246, 2010.

[13] OpenStreetMap Contributors. Nominatim geocoding api and openstreetmap spatial data documentation. https://nominatim.org/release-docs/latest/, 2024.

[14] PingCAP Inc. Tidb cloud distributed sql database documentation. https://docs.pingcap.com/tidbcloud/, 2024.

[15] Sebastián Ramírez. Fastapi: High-performance asynchronous web framework for python. https://fastapi.tiangolo.com/, 2024.

[16] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill Education, 7 edition, 2020.

[17] Roger W. Sinnott. Virtues of the haversine. *Sky and Telescope*, 68(2):159, 1984.

[18] Evan You and Vite Contributors. Vite: Next generation frontend tooling. https://vitejs.dev/, 2024.